

## SECTION 9

# MEMORY MANAGEMENT UNIT

The MC68030 includes a memory management unit (MMU) that supports a demand-paged virtual memory environment. The memory management is “demand” in that programs do not specify required memory areas in advance, but request them by accessing logical addresses. The physical memory is paged, meaning that it is divided into blocks of equal size called page frames. The logical address space is divided into pages of the same size. The operating system assigns pages to page frames as they are required to meet the needs of programs.

The principal function of the MMU is the translation of logical addresses to physical addresses using translation tables stored in memory. The MMU contains an address translation cache (ATC) in which recently used logical-to-physical address translations are stored. As the MMU receives each logical address from the CPU core, it searches the ATC for the corresponding physical address. When the translation is not in the ATC, the processor searches the translation tables in memory for the translation information. The address calculations and bus cycles required for this search are performed by microcode and dedicated logic in the MC68030. In addition, the MMU contains two transparent translation registers (TT0 and TT1) that identify blocks of memory that can be accessed without translation. The features of the MMU are:

- 32-Bit Logical Address Translated to 32-Bit Physical Address with 3-Bit Function Code
- Supports Two-Clock Cycle Processor Accesses to Physical Address Spaces
- Addresses Translated in Parallel with Accesses to Data and Instruction Caches
- On-Chip Fully Associative 22-Entry ATC
- Translation Table Search Controlled by Microcode
- Eight Page Sizes: 256, 512, 1K, 2K, 4K, 8K, 16K and 32K Bytes
- Separate User and Supervisor Translation Table Trees Are Supported
- Two Independent Blocks Can Be Defined as Transparent (Untranslated)
- Multiple Levels of Translation Tables

- 0–15 Upper Logical Address Bits Can Be Ignored (Using Initial Shift)
- Portions of Tables Can Be Undefined (Using Limits)
- Write Protection and Supervisor Protection
- History Bits Automatically Maintained in Page Descriptors
- Cache Inhibit Output ( $\overline{\text{CIOUT}}$ ) Signal Asserted on Page Basis
- External Translation Disable Input Signal ( $\overline{\text{MMUDIS}}$ )
- Subset of Instruction Set Defined by MC68851

The MMU completely overlaps address translation time with other processing activity when the translation is resident in the ATC. ATC accesses operate in parallel with the on-chip instruction and data caches.

Figure 9-1 is a block diagram of the MC68030 showing the relationship of the MMU to the execution unit and the bus controller. For an instruction or operand access, the MC68030 simultaneously searches the caches and searches for a physical address in the ATC. If the translation is available, the MMU provides the physical address to the bus controller and allows the bus cycle to continue. When the instruction or operand is in either of the on-chip caches on a read cycle, the bus controller aborts the bus cycle before address strobe is asserted. Similarly, the MMU causes a bus cycle to abort before the assertion of address strobe when a valid translation is not available in the ATC or when an invalid access is attempted.

An MMU disable input signal ( $\overline{\text{MMUDIS}}$ ) is provided that dynamically disables address translation for emulation, diagnostic, or other purposes.

The programming model of the MMU (see Figure 9-2) consists of two root pointer registers, a control register, two transparent translation registers, and a status register. These registers can only be accessed by supervisor programs. The CPU root pointer register points to an address translation tree structure in memory that describes the logical-to-physical mapping for user accesses or for both user and supervisor accesses. The supervisor root pointer register optionally points to an address translation tree structure for supervisor mappings. The translation control register is comprised of fields that control the translation operation. Each transparent translation register can define a block of logical addresses that are used as physical addresses (without translation). The MMU status register contains accumulated status information from a translation performed as a part of a PTEST instruction.

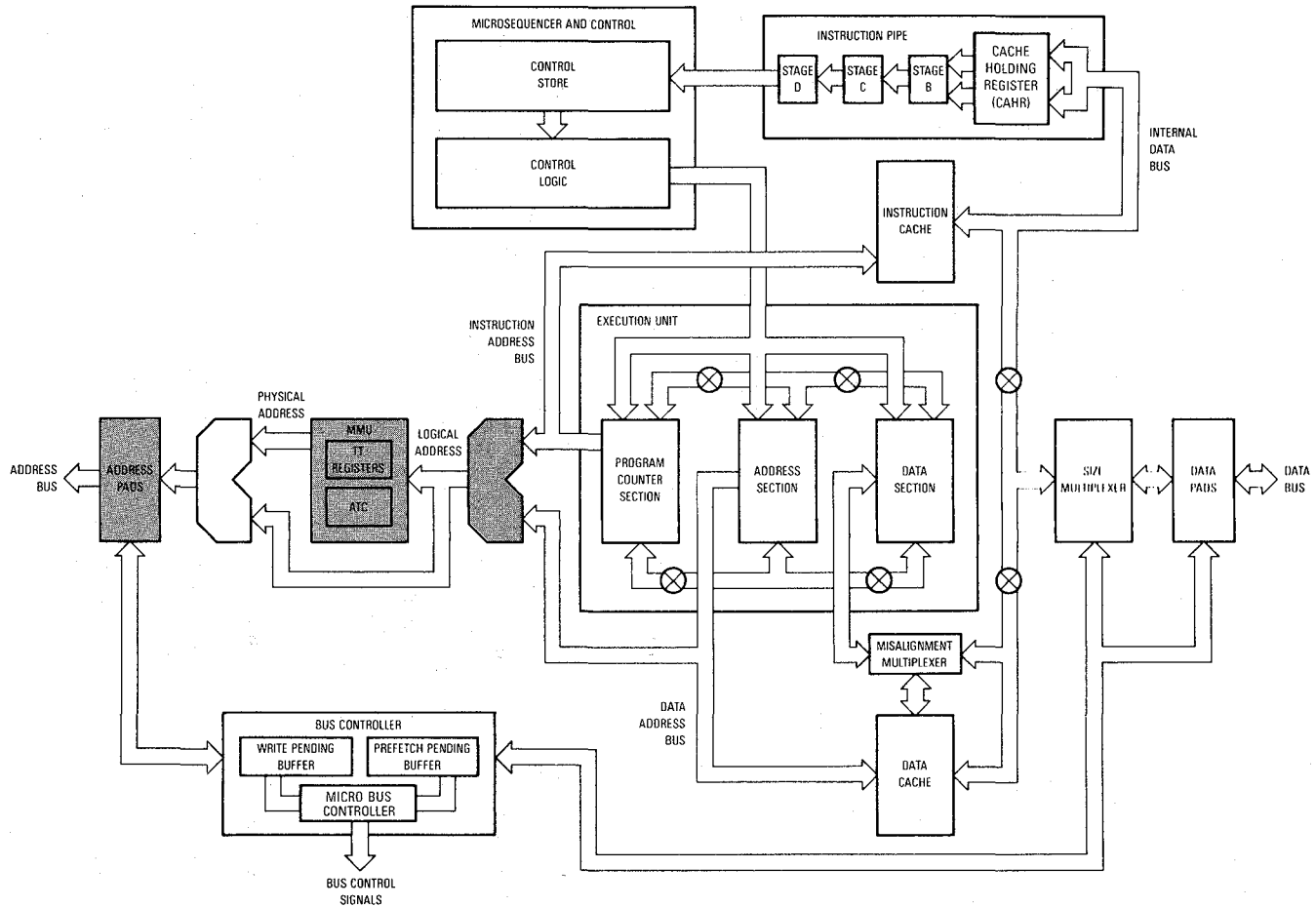
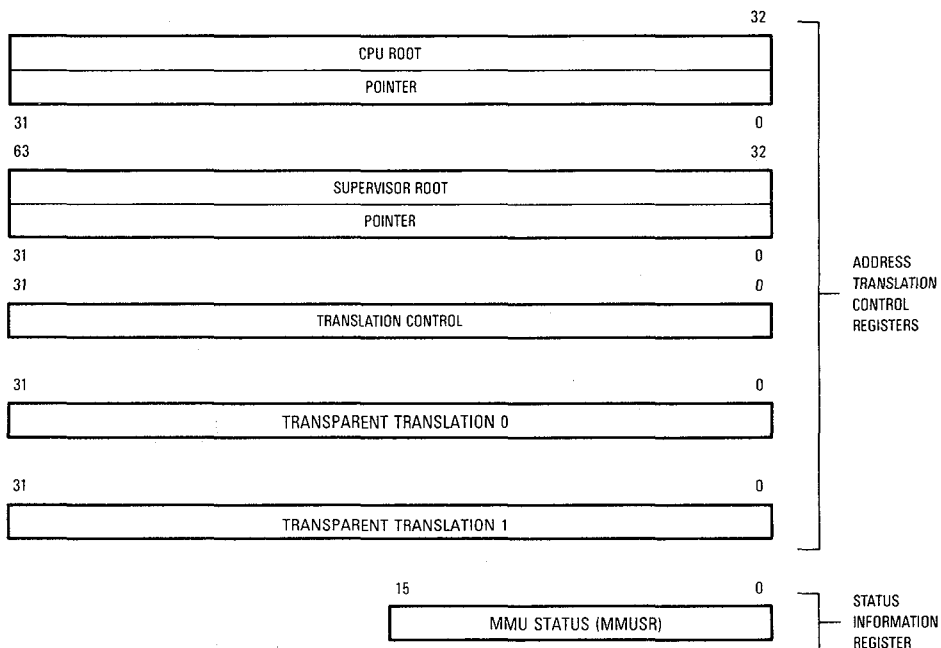


Figure 9-1. MMU Block Diagram



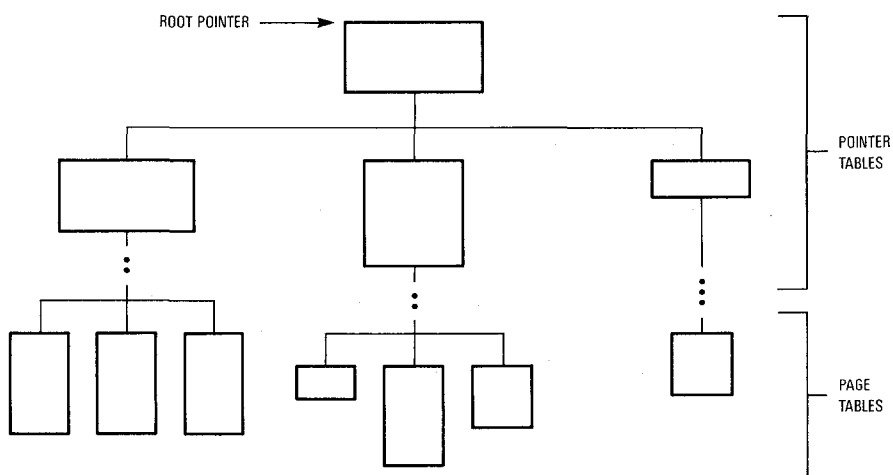
**Figure 9-2. MMU Programming Model**

The ATC in the MMU is a fully associative cache that stores 22 logical-to-physical address translations and associated page information. It compares the logical address and function code internally supplied by the processor with all tag entries in the ATC. When the access address and function code matches a tag in the ATC (a hit occurs) and no access violation is detected, the ATC outputs the corresponding physical address to the bus controller, which continues the external bus cycle. Function codes are routed to the bus controller unmodified.

Each ATC entry contains a logical address, a physical address, and status bits. Among the status bits are the write protect and cache inhibit bits.

When the ATC does not contain the translation for a logical address (a miss occurs) and an external bus cycle is required, the MMU aborts the access and causes the processor to initiate bus cycles that search the translation tables in memory for the correct translation. If the table search completes without any errors, the MMU stores the translation in the ATC and provides the physical address for the access, allowing the bus controller to retry the original bus cycle.

An MMU translation table has a tree structure with the base of the first table defined by a root pointer descriptor. The root pointer descriptor of the current translation table is resident in one of two root pointer registers. The general tree structure is shown in Figure 9-3. Table entries at the upper levels of a tree point to other tables. The table leaf entries are page frame addresses. All addresses stored in the translation tables are physical addresses; the translation tables reside in the physical address space.



**Figure 9-3. Translation Table Tree**

System software selects the parameters for the translation tables by configuring the translation control register (TC) appropriately. The function codes or a portion of the logical address can be defined as the index into the first level of lookup in the table. The TC register specifies how many bits of the logical address are used as the index for each level of the lookup (as many as 15 bits can be used at a given level).

## 9.1 TRANSLATION TABLE STRUCTURE

The M68030 uses the ATC and translation tables stored in memory to perform the translation from a logical to a physical address. Translation tables for a program are loaded into memory by the operating system.

The general translation table structure supported by the MC68030 is a tree structure of tables. The pointer tables form the branches of the tree. These tables contain the base addresses of other tables. Page descriptors can reside in pointer tables and, in that case, are called early termination descriptors. The tables at the leaves of the tree are called page tables. Only a portion of the translation table for the entire logical address space is required to be resident in memory at any time: specifically, only the portion of the table that translates the logical addresses that the currently executing process(es) use(s) must be resident. Portions of translation tables can be dynamically allocated as the process requires additional memory.

As shown in Figure 9-4, the root pointer for a table is a descriptor that contains the base address of the top level table for the tree. The pointer tables and page tables also consist of descriptors. A descriptor in a pointer table typically contains the base address of a table at the next level of the tree. A table descriptor can also contain limits for the index into the next table, protection information, and history information pertaining to the descriptor. Each table is indexed by a field extracted from the logical address. In the example shown in Figure 9-4, the A field of the logical address, \$00A, is added to the root pointer value to select a descriptor at the A level of the translation tree. The selected descriptor points to the base of the appropriate page table, and the B field of the logical address (\$006) is added to this base address to select a descriptor within the page table. A descriptor in a page table contains the physical base address of the page, protection information, and history information for the page. A page descriptor can also reside in a pointer table or even in a root pointer to define a contiguous block of pages. A two-level page task is shown. The 32-bit logical address space is divided into 4096 segments of 1024 bytes each.

Figure 9-5 shows a possible layout of this example translation tree in memory.

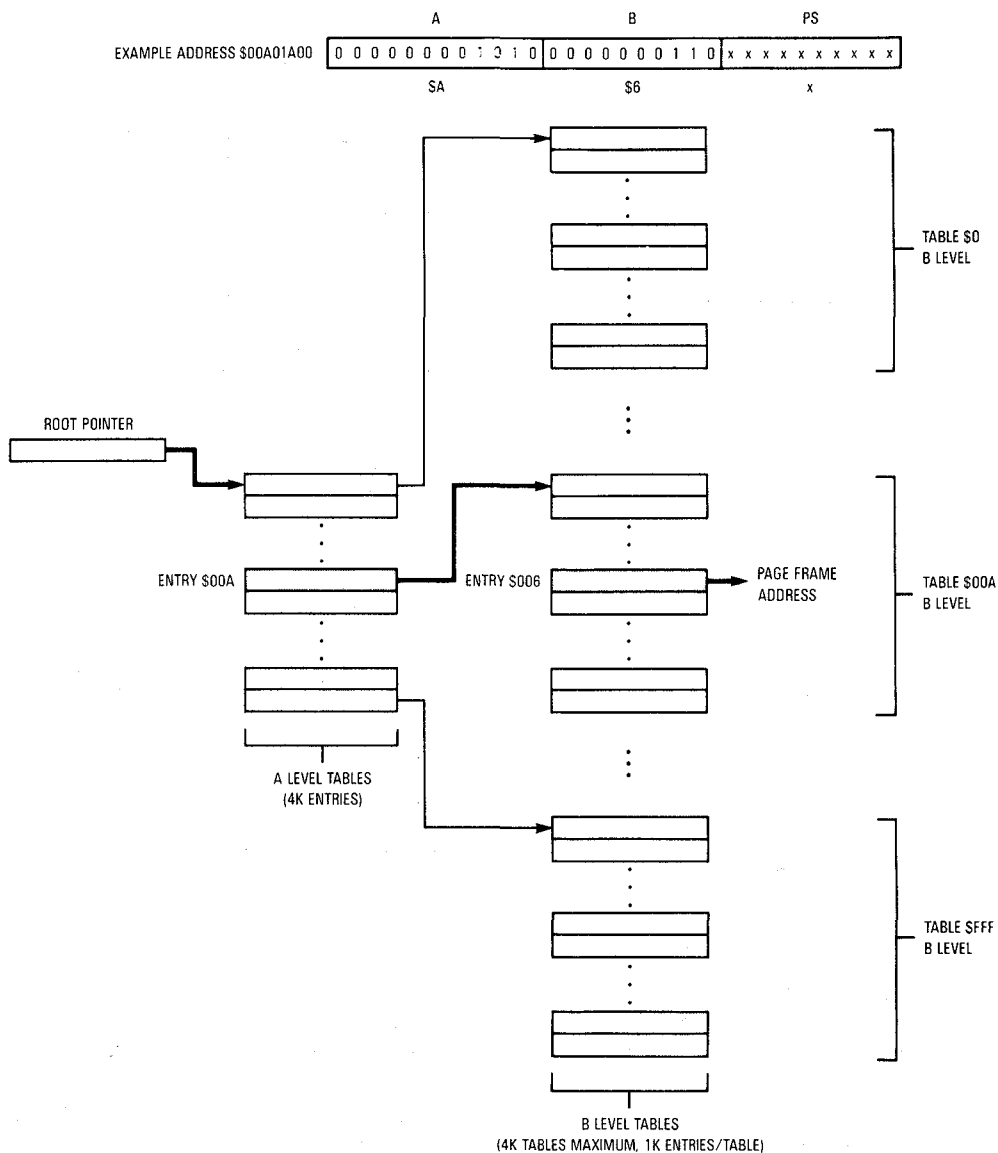
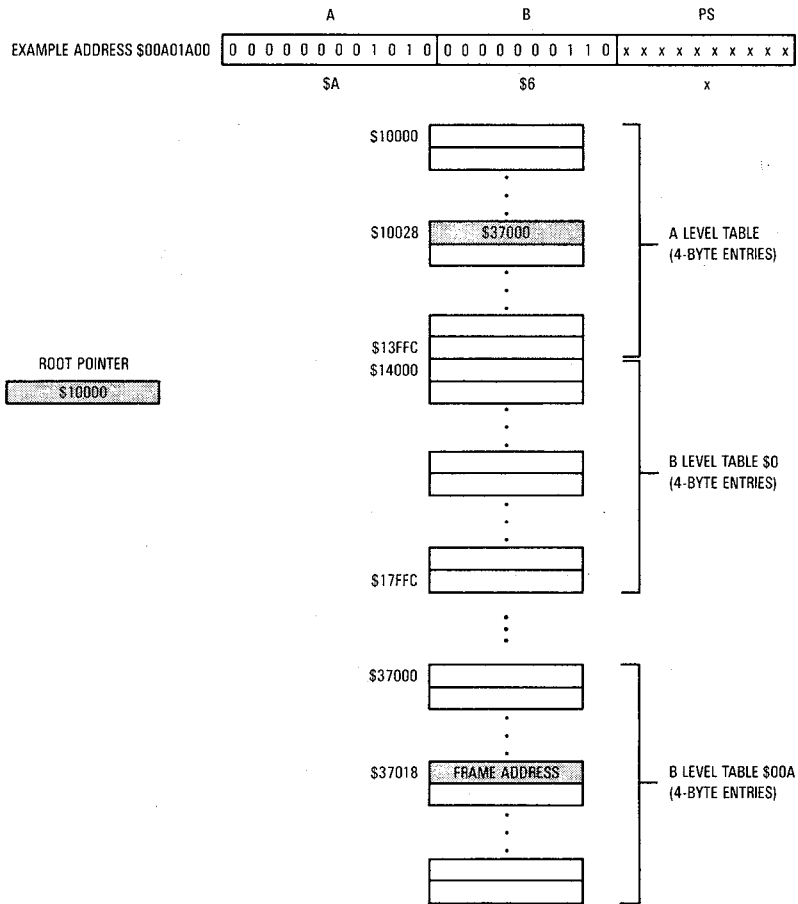


Figure 9-4. Example Translation Table Tree



**Figure 9-5. Example Translation Tree Layout in Memory**

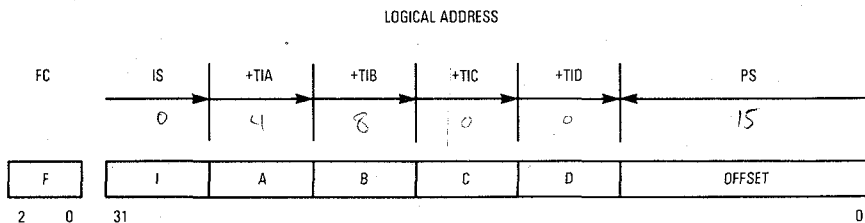
### 9.1.1 Translation Control

The translation control register (TC) defines the size of pages in memory, selects the root pointer register to be used for supervisor accesses, indicates whether the top level of the translation tree is indexed by function code, and specifies the number of logical address bits used to index into the various levels of the translation tree. The initial shift (IS) field of the TC register defines the size of the logical address space; it contains the number of most significant address bits that are ignored in the translation table lookup. For example, if the IS field is set to zero, the logical address space is  $2^{32}$  bytes. On the other hand, if the IS field is set to 15, the logical address space contains only  $2^{32} - 2^{15}$  bytes.

The page size (PS) field of the TC register specifies the page size for the system. The number of pages in the system is equal to the logical address space divided by the page size. The maximum number of pages that can be defined by a translation tree is greater than 16 million ( $2^{32}/2^8$ ). The minimum number is 4 ( $2^{17}/2^{15}$ ). The function code can also be used in the table lookup, defining as many as seven regions of the above size ( $FC = 0-6$ ). The entire range of the logical address space(s) can be defined by translation tables of many sizes. The MC68030 provides flexibility that simplifies the implementation of large translation tables.

The use of a tree structure with as many as five levels of tables provides granularity in translation table design. The LIMIT field of the root pointer can limit the value of the first index and limits the actual number of descriptors required. Optionally, the top level of the structure can be indexed by function code bits. In this case, the pointer table at this level contains eight descriptors. The next level of the structure (or the top level when the FCL bit of the TC register is set to zero) is indexed by the most significant bits of the logical address (disregarding the number of bits specified by the IS field). The number of logical address bits used for this index is specified by the TIA field of the TC register. If, for example, the TIA field contains the value 5, the index for this level contains five bits, and the pointer table at this level contains at most 32 descriptors.

Similarly, the TIB, TIC, and TID fields of the TC register define the indexes for lower levels of the translation table tree. When one of these fields contains zero, the remaining Tlx fields are ignored; the last nonzero Tlx field defines the index into the lowest level of the tree structure. The tables selected by the index at this level are page tables; every descriptor in these tables is (or represents) a page descriptor. Figure 9-6 shows how the Tlx fields of the TC register apply to a function code and logical address.



**Figure 9-6. Derivation of Table Index Fields**

For example, a TC register in which the FCL bit is set to one, the TIA field contains five, the TIB field contains nine, and the TIC and TID fields contain zero defines a three-level translation tree. The top level is indexed by the function code, the next level by five logical address bits, and the bottom level by nine logical address bits. If the TIC field contained nine instead of zero, the translation tree would have four levels, and the two bottom levels would each be indexed by 9-bit portions of the logical address.

The following equation for fields in the TC register must be satisfied:

$$IS + PS + TIA + TIB^1 + TIC^1 + TID^1 = 32$$

That is, every bit of the logical address either addresses a byte within the page, is part of the index at some level of the address table, or is explicitly ignored by initial shift.

Table 9-1 lists the valid sizes of the table indexes at each of the levels indexed by the Tlx fields and the position of each table index within the logical address. When the function code is also used to select a descriptor, a total of five levels can be defined by the logical address. The function code lookup level and levels B, C, and D can be suppressed.

**Table 9-1. Size Restrictions**

| Field | Starting Bit Position | Size Restrictions                                               |
|-------|-----------------------|-----------------------------------------------------------------|
| A     | 31-IS                 | 1–15 (TIA must be greater than zero; minimum of two if TIB = 0) |
| B     | 31-IS-TIA             | 0–15                                                            |
| C     | 31-IS-TIA-TIB         | 0–15 (ignored if TIB is zero)                                   |
| D     | 31-IS-TIA-TIB-TIC     | 0–15 (ignored if TIB or TIC is zero)                            |

### 9.1.2 Translation Table Descriptors

The address translation trees consist of tables of descriptors. These descriptors can be one of four basic types: table descriptors, page descriptors (normal or early termination), invalid descriptors, or indirect descriptors. Each of these descriptors has both a long-format and a short-format representation.

A root pointer descriptor defines the root of a tree and can be a table descriptor or an early termination page descriptor. A table descriptor points to a descriptor table in memory that defines the next lower level in the translation tree. An early termination page descriptor causes immediate termi-

NOTE 1: If any of these fields are zero, the remaining fields are ignored.

nation of the table search and contains the physical address of an area in memory that contains page frames corresponding to contiguous logical addresses (Refer to **9.5.3.1 EARLY TERMINATION AND CONTIGUOUS MEMORY**).

Tables at intermediate levels of a translation tree contain descriptors that are similar to the root pointer descriptors. They can contain table descriptors or early termination page descriptors and can also contain invalid descriptors.

The descriptor tables at the lowest level of a translation tree can only contain page descriptors, indirect descriptors, and invalid descriptors. A page descriptor in the lowest level of a translation tree defines the physical address of a page frame in memory that corresponds to the logical address of a page. An indirect descriptor contains a pointer to the actual page descriptor and can be used when a single page descriptor is accessed by two or more logical addresses.

To enhance the flexibility of translation table design, descriptors (except for root pointer descriptors) can be either short or long format. The short-format descriptors consist of one long word and have no index-limiting capabilities or supervisor-only protection. The long-format descriptors consist of two long words and contain all defined descriptor fields for the MC68030. The pointer and page tables can each contain either short- or long-format descriptors, but no single table can contain both sizes. Tables at different levels of the translation tree can contain different formats of descriptors. Tables at the same level can also contain descriptors of different formats, but all descriptors in a particular pointer table or page table must be of the same format. Figure 9-7 shows a translation tree that uses several different format descriptors.

All descriptors contain a descriptor type (DT) field, which identifies the descriptor or specifies the size of the descriptors in the table to which the descriptor points. It is always the two least significant bits of the most significant (or only) long word of a descriptor.

Invalid descriptors can be used at any level of the translation tree except the root. When a table search for a normal translation encounters an invalid descriptor, the processor takes a bus error exception. The invalid descriptor can be used to identify either a page or branch of the tree that has been stored on an external device and is not resident in memory or a portion of the translation table that has not yet been defined. In these two cases, the exception routine can either restore the page from disk or add to the translation table.



All long-format descriptors and short-format invalid descriptors include one or two unused fields. The operating system can use these fields for its own purposes. For example, the operating system can encode these fields to specify the type of invalid descriptor. Alternately, the external device address of a page that is not resident in main memory can be stored in the unused field.

## 9.2 ADDRESS TRANSLATION

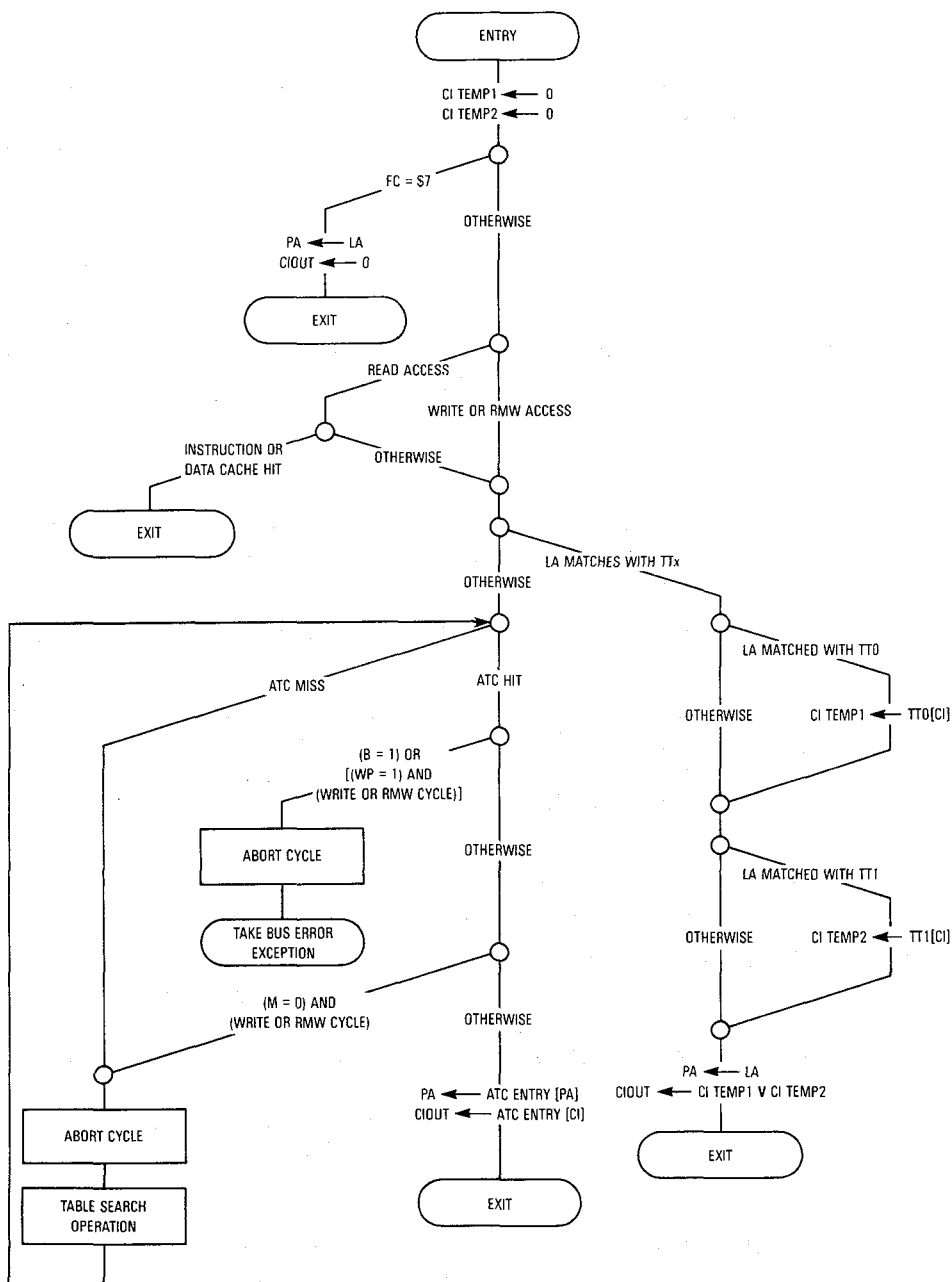
The function of the MMU is to translate logical addresses to physical addresses according to control information stored by the system in the MMU registers and in translation table trees resident in memory.

### 9.2.1 General Flow for Address Translation

A CPU space address ( $FC0-FC2 = \$7$ ) is a special case that is immediately used as a physical address without translation. For other accesses, the translation process proceeds as follows:

1. Search the on-chip data and instruction caches for the required instruction word or operand on read accesses.
2. Compare the logical address and function code to the transparent translation parameters in the transparent translation registers, and use the logical address as a physical address for the memory access when one or both of the registers match.
3. Compare the logical address and function code to the tag portions of the entries in the ATC and use the corresponding physical address for the memory access when a match occurs.
4. When no on-chip cache hit occurs (on a read) and no TTx register matches or valid ATC entry matches, initiate a table search operation to obtain the corresponding physical address from the corresponding translation tree, create a valid ATC entry for the logical address, and repeat step 3.

Figure 9-8 is a general flowchart for address translation. The top branch of the flowchart applies to CPU space accesses ( $FC0-FC2 = \$7$ ). The next branch applies to read accesses only. When either of the on-chip caches hits (contains the required data or instruction), no memory access is necessary. The third branch applies to transparent translation. The bottom three branches apply to ATC translation as follows. If the requested access misses in the ATC, the memory cycle is aborted, and a table search operation proceeds. An ATC entry is created after the table search, and the access is retried. If an access



**Figure 9-8. Address Translation General Flowchart**

hits in the ATC but a bus error was detected during the table search that created the ATC entry, the memory access is aborted, and a bus error exception is taken.

If an access results in an ATC hit but the access is either a write or read-modify-write access and the page is write protected, the memory cycle is also aborted, and a bus error exception is taken. For a write or read-modify-write access, when the modified bit of the ATC entry is not set, the memory cycle is aborted, a table search proceeds to set the modified bit in both the page descriptor in memory and in the ATC, and the access is retried. If the modified bit of the ATC entry is set and the bus error bit is not, assuming that neither TT<sub>x</sub> register matches and the access is not to CPU space, the ATC provides the address translation to the bus controller under two conditions: 1) if a read access does not hit in either on-chip cache and 2) if a write or read-modify-write access is not write protected.

The preceding description of the general flowchart specifies several conditions that cause the memory cycle to be aborted. In these cases, the bus cycle is aborted before the assertion of  $\overline{AS}$ .

## 9.2.2 Effect of $\overline{RESET}$ on MMU

When the MC68030 is reset by the assertion of the  $\overline{RESET}$  signal, the E bits of the TC and TT<sub>x</sub> registers are cleared, disabling address translation. This causes logical addresses to be passed through as physical addresses to the bus controller, allowing an operating system to set up the translation tables and MMU registers, as required. After it has initialized the translation tables and registers, the E bit of the TC register can be set, enabling address translation. A reset of the processor does not invalidate any entries in the ATC. An MMU instruction (such as PMOVE) that flushes the ATC must be executed to flush all existing valid entries from the ATC after a reset operation and before translation is enabled.

## 9.2.3 Effect of $\overline{MMUDIS}$ on Address Translation

The assertion of  $\overline{MMUDIS}$  prevents the MMU from performing searches of the ATC and the execution unit from performing table searches. With address translation disabled, logical addresses are used as physical addresses. When an initial access to a long-word-aligned data operand that is larger than the addressed port size is performed, the successive bus cycles for additional portions of the operand always use the same higher order address bits that were used for the initial bus cycle (changing A0 and A1 appropriately). Thus,

if `MMUDIS` is asserted during this type of operation, the disabling of address translation does not become effective until the entire transfer is complete. Note that the assertion of `MMUDIS` does not affect the operation of the transparent translation registers.

## 9.3 TRANSPARENT TRANSLATION

Two independent transparent translation registers (TT0 and TT1) in the MMU optionally define two blocks of the logical address space that are directly translated to the physical address spaces. The MMU does not explicitly check write protection for the addresses in these blocks, but a block can be specified as transparent only for read cycles. The blocks of addresses defined by the TTx registers include at least 16M bytes of logical address space; the two blocks can overlap, or they can be separate.

The following description of the address comparison assumes that both TT0 and TT1 are enabled; however, each TTx register can be independently disabled. A disabled TTx register is completely ignored.

When the MMU receives an address to be translated, the function code and the eight high-order bits of the address are compared to the block of addresses defined by TT0 and TT1. The address space block for each TTx register is defined by the base function code, the function code mask, the logical base address, and the logical address mask. When a bit in a mask field is set, the corresponding bit of the base function code or logical base address is ignored in the function code and address comparison. Setting successively higher order bits in the address mask increases the size of the transparently translated block.

The address for the current bus cycle and a TTx register address match when the function code bits and address bits (not including masked bits) are equal. Each TTx register can specify read accesses or write accesses as transparent. In that case, the internal read/write signal must match the R/W bit in the TTx register for the match to occur. The selection of the type of access (read or write) can also be masked. The read/write mask bit (RWM) must be set for transparent translation of addresses used by instructions that execute read-modify-write operations. Otherwise, neither the read nor write portions of read-modify-write operations are mapped transparently with the TTx registers, regardless of the function code and address bits for the individual cycles within a read-modify-write operation.

By appropriately configuring a transparent translation register, flexible transparent mapping can be specified. For instance, to transparently translate user program space with a TTx register, the RWM bit of the register is set to 1, the FC BASE is set to \$2, and the FC MASK is set to \$0. To transparently translate supervisor data read accesses of addresses \$00000000–\$0FFFFFFF, the LOGICAL BASE ADDRESS field is set to \$0X, the LOGICAL ADDRESS MASK is set to \$0F, the R/W bit is set to 1, the RWM bit is set to 0, the FC BASE is set to \$5, and the FC MASK field is set to \$0. Since only read cycles are specified by the TTx register for this example, write accesses for this address range can be translated with the translation tables and write protection can be implemented as required.

Each TTx register can specify that the contents of logical addresses in its block should not be stored in either an internal or external cache. The cache inhibit out signal ( $\overline{\text{CIOUT}}$ ) is asserted when an address matches the address specified by a TTx register and the cache inhibit bit in that TTx register is set.  $\overline{\text{CIOUT}}$  is used by the on-chip instruction and data caches to inhibit caching of data associated with this address. The signal is available to external caches for the same purpose.

For an access, if either of these registers match, the access is transparently translated. If both registers match, the CI bits are ORed together to generate the  $\overline{\text{CIOUT}}$  signal.

Transparent translation can also be implemented by the translation tables of the translation trees if the physical addresses of pages are set equal to the logical addresses.

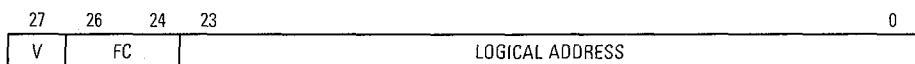
## 9.4 ADDRESS TRANSLATION CACHE

The ATC is a 22-entry fully associative (content addressable) cache that contains address translations in a form similar to the corresponding page descriptors in memory to provide fast address translation of a recently used logical address.

The MC68030 is organized such that the translation time of the ATC is always completely overlapped by other operations; thus, no performance penalty is associated with ATC searches. The address translation occurs in parallel with on-chip instruction and data cache accesses before an external bus cycle begins.

If possible, when the ATC stores a new address translation, it replaces an entry that is no longer valid. When all entries in the ATC are valid, the ATC selects a valid entry to be replaced, using a pseudo least recently used algorithm. The ATC uses a validity bit and an internal history bit to implement this replacement algorithm. ATC hit rates are application dependent, but hit rates ranging from 98% to greater than 99% can be expected.

Each ATC entry consists of a logical address and information from a corresponding page descriptor that contains the physical address. The 28-bit logical (or tag) portion of each entry consists of three fields:



#### V — VALID

This bit indicates the validity of the entry. If V is set, this entry is valid. This bit is set when the MC68030 loads an entry. A flush operation clears the bit. Specifically, any of these operations clear the V bit of an entry:

- A PMOVE instruction with the FD bit equal to zero that loads a value into the CRP, SRP, TC, TT0, or TT1 register.
- A PFLUSHA instruction.
- A PFLUSH instruction that selects this entry.
- A PLOAD instruction for a logical address and FC that matches the tag for this entry. The instruction writes a new entry (with the V bit set) for the specified logical address.
- The selection of this entry for replacement by the replacement algorithm of the ATC.

#### FC — FUNCTION CODE

This 3-bit field contains the function code bits (FC0–FC2) corresponding to the logical address in this entry.

#### LOGICAL ADDRESS

This 24-bit field contains the most significant logical address bits for this entry. All 24 bits of this field are used in the comparison of this entry to an incoming logical address when the page size is 256 bytes. For larger page sizes, the appropriate number of least significant bits of this field are ignored.

Each logical portion of an entry has a corresponding 28-bit physical (or data) portion. The physical portion contains these fields:

|    |    |    |    |                  |   |
|----|----|----|----|------------------|---|
| 27 | 26 | 25 | 24 | 23               | 0 |
| B  | CI | WP | M  | PHYSICAL ADDRESS |   |

#### B — BUS ERROR

This bit is set for an entry if a bus error, an invalid descriptor, a supervisor violation, or a limit violation is encountered during the table search corresponding to this entry. When B is set, a subsequent access to the logical address causes the MC68030 to take a bus error exception. Since an ATC miss causes an immediate retry of the access after the table search operation, the bus error exception is taken on the retry. The B bit remains set until a PFLUSH instruction or a PLOAD instruction for this entry invalidates the entry or until the replacement algorithm for the ATC replaces it.

#### CI — CACHE INHIBIT

This bit is set when the cache inhibit bit of the page descriptor corresponding to this entry is set. When the MC68030 accesses the logical address of an entry with the CI bit set, it asserts the cache inhibit out signal (CIOUT) during the corresponding bus cycle. This signal inhibits caching in the on-chip caches and can also be used for external caches.

#### WP — WRITE PROTECT

This bit is set when a WP bit is set in any of the descriptors encountered during the table search for this entry. Setting a WP bit in a table descriptor write protects all pages accessed with that descriptor. When the WP bit is set, a write access or a read-modify-write access to the logical address corresponding to this entry causes a bus error exception to be taken immediately.

#### M — MODIFIED

This bit is set when a valid write access to the logical address corresponding to the entry occurs. If the M bit is clear and a write access to this logical address is attempted, the MC68030 aborts the access and initiates a table search, setting the M bit in the page descriptor, invalidating the old ATC entry, and creating a new entry with the M bit set. The MMU then allows the original write access to be performed. This assures that the first write operation to a page sets the M bit in both the ATC and the page descriptor in the translation tables even when a previous read operation to the page had created an entry for that page in the ATC with the M bit clear.

## PHYSICAL ADDRESS

This 24-bit field contains the physical address bits (A31–A8) from the page descriptor corresponding to the logical address. When the page size is larger than 256 bytes, not all bits in the physical address field are used. All page index bits of the logical address are transferred to the bus controller without translation.

## 9.5 TRANSLATION TABLE DETAILS

The details of translation tables and their use include detailed descriptions of the descriptors, table searching, translation table structure variations, and the protection techniques available with the MC68030 MMU.

### 9.5.1 Descriptor Details

The descriptor details include detailed descriptions of the short- and long-format descriptors used in the translation trees. The fields that apply to all descriptors are described in the first paragraph.

**9.5.1.1 DESCRIPTOR FIELD DEFINITIONS.** All descriptor fields are used in more than one type of descriptor. This section lists these fields and describes the use of each field.

#### DT

This 2-bit field contains the descriptor type; the first two types apply to the descriptor itself. The other two types apply to the descriptors in the table at the next level of the tree. The values are defined as follows:

- \$0    **INVALID**  
This code identifies the current descriptor as an invalid descriptor. A table search ends when an invalid descriptor is encountered.
- \$1    **PAGE DESCRIPTOR**  
This code identifies the current descriptor as a page descriptor. The page descriptor is a normal page descriptor when it resides in a page table (in the bottom level of the translation tree). A page descriptor at a higher level is an early termination page descriptor. A table search ends when a page descriptor of either type is encountered.

## \$2 VALID 4 BYTE

This code specifies that the next table to be accessed contains short-format descriptors. The MC68030 multiplies the index for the next table by four to access the next descriptor. (Short-format descriptors must be long-word aligned.) When used in a page table (bottom level of an translation tree), this code identifies an indirect descriptor that points to a short-format page descriptor.

## \$3 VALID 8 BYTE

This code specifies that the next table to be accessed contains long-format descriptors. The MC68030 multiplies the index for the next table by eight to access the next descriptor. (Long-format descriptors must be quad-word aligned.) When used in a page table (bottom level of an address translation tree), this code identifies an indirect descriptor that points to a long-format page descriptor.

## U

This bit is automatically set by the processor when a descriptor is accessed in which the U bit is clear except after a supervisor violation is detected. In a page descriptor table, this bit is set to indicate that the page corresponding to the descriptor has been accessed. In a pointer table, this bit is set to indicate that the pointer has been accessed by the MC68030 as part of a table search. Note that a pointer may be fetched, and its U bit set, for an address to which access is denied at another level of the tree. Updates of the U bit are performed before the MC68030 allows a page to be accessed. The processor never clears this bit.

## WP

This bit provides write protection. The states of all WP bits encountered during a table search are logically ORed, and the result is copied to the ATC entry at the end of a table search for a logical address. During a table search for a PTEST instruction, the processor copies this result into the MMU status register (MMUSR). When WP is set, the MC68030 does not allow the logical address space mapped by that descriptor to be written by any program (i.e., this protection is absolute). If the WP bit is clear, the MC68030 allows write accesses using this descriptor (unless access is restricted at some other level of the translation tree).

## CI

This bit is set to inhibit caching of items within this page by the on-chip instruction and data caches and, also, to cause the assertion of the CIOUT signal by the MC68030 for bus cycles accessing items within this page.

## L/U

This bit specifies the type of limit in the LIMIT field. When the L/U bit is set, the LIMIT field contains the unsigned lower limit; the index value for the next level of the tree must be greater than or equal to the value in the LIMIT field. When the bit is cleared, the limit is an unsigned upper limit, and the index value must be less than or equal to the LIMIT. An out-of-bounds access causes the B bit in the ATC entry for the address to be set and causes the table search to abort.

## LIMIT

This 15-bit field contains a limit to which the index portion of an address is compared to detect an out-of-bounds index. The limit check applies to the index into the table at the next lower level of the translation tree. If the descriptor is an early termination page descriptor, the limit field is still used as a check on the next index field of the logical address.

## M

This bit identifies a modified page. The MC68030 sets the M bit in the corresponding page descriptor before a write operation to a page for which the M bit is zero, except after a descriptor with the WP bit set is encountered, or after a supervisor violation is encountered. An access is considered to be a write for updating purposes if either the R/W or RMC signal is low. The MC68030 never clears this bit.

# 9

## PAGE ADDRESS

This 24-bit field contains the physical base address of a page in memory. The low-order bits of the address are supplied by the logical address. When the page size is larger than 256 bytes, one or more of the least significant bits of this field are not used. The number of unused bits is equal to the PS field value in the TC register minus eight.

## S

This bit identifies a pointer table or a page as a supervisor only table or page. When the S bit is set, only programs operating at the supervisor privilege level are allowed to access the portion of the logical address mapped by this descriptor. If this bit is clear, accesses using this descriptor are not restricted to supervisor-only unless the access is restricted by some other level of the translation tree.

## TABLE ADDRESS

This 28-bit field contains the physical base address of a table of descriptors. The low-order bits of the address are supplied by the logical address.

## DESCRIPTOR ADDRESS

This 30-bit field, which contains the physical address of a page descriptor, is only used in short- and long-format indirect descriptors.

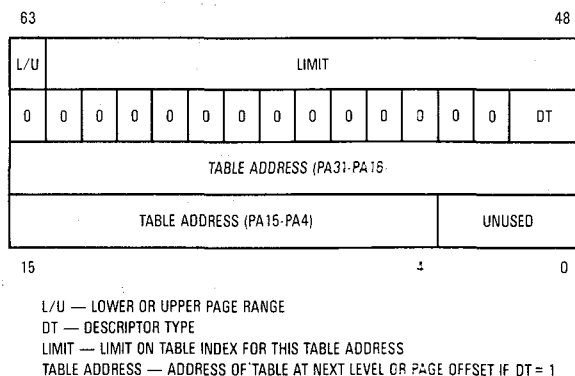
## UNUSED

The bits in this field are not used by the MC68030 and may be used by the system software.

## RESERVED

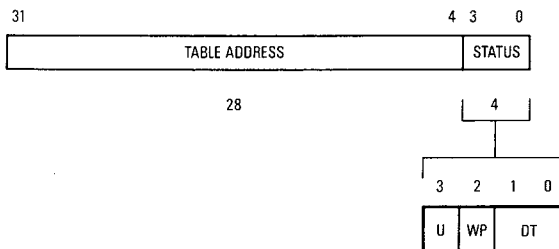
Descriptor fields designated by a one or a zero are reserved by Motorola for future definition. These bits should be consistently written as either a one or a zero as appropriate. In the root pointers, these bits are not alterable. In memory-resident descriptors, the values in these fields are neither checked nor altered by the MC68030. Use of these bits by system software for any purpose may not be supported in future products.

**9.5.1.2 ROOT POINTER DESCRIPTOR.** A root pointer descriptor contains the address of the top-level pointer table of a translation tree. This type of descriptor is loaded into the CRP and SRP registers with the PMOVE instruction. The field descriptions in the preceding section apply to corresponding fields of the CRP and SRP with two minor exceptions. A descriptor-type code of \$00 (invalid) is not allowed; an attempt to load zero into the DT field of the CRP or SRP register results in an MMU configuration exception. Also, when the FCL field of the TC register is set, the L/U and LIMIT fields of the root pointer registers are unused. Figure 9-9 shows the root pointer descriptor format.



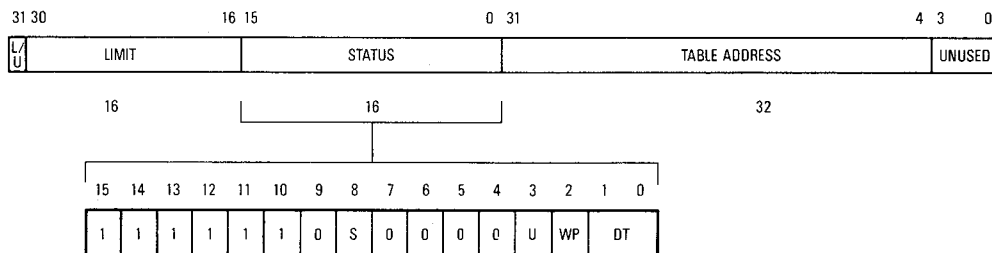
**Figure 9-9. Root Pointer Descriptor Format**

**9.5.1.3 SHORT-FORMAT TABLE DESCRIPTOR.** The field descriptions in **9.5.1.1 DESCRIPTOR FIELD DEFINITIONS** apply to corresponding fields of this descriptor. Figure 9-10 shows the format of the short-format table descriptor.



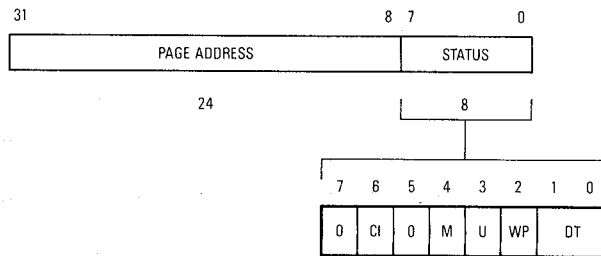
**Figure 9-10. Short-Format Table Descriptor**

**9.5.1.4 LONG-FORMAT TABLE DESCRIPTOR.** The field descriptions in **9.5.1.1 DESCRIPTOR FIELD DEFINITIONS** apply to corresponding fields of this descriptor. During address computations, the MC68030 internally replaces the UNUSED field with zeros. Figure 9-11 shows the format of the long-format table descriptor.



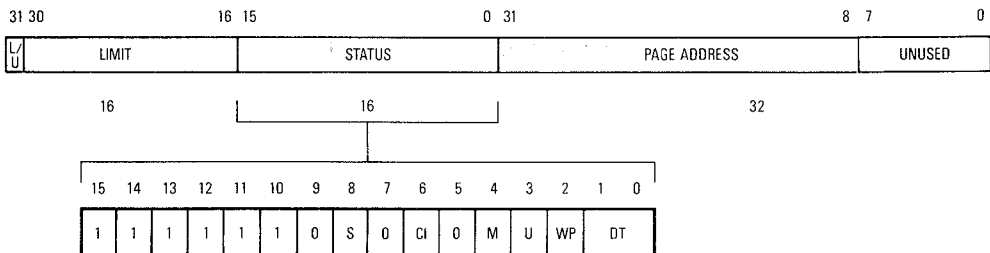
**Figure 9-11. Long-Format Table Descriptor**

**9.5.1.5 SHORT-FORMAT EARLY TERMINATION PAGE DESCRIPTOR.** The short-format early termination page descriptor contains the page descriptor code in the DT field but resides in a pointer table. That is, the table in which an early termination page descriptor is located is not at the bottom level of the address translation tree. The field descriptions in **9.5.1.1 DESCRIPTOR FIELD DEFINITIONS** apply to corresponding fields of this descriptor. Figure 9-12 shows the format of the short-format early termination page descriptor.



**Figure 9-12. Short-Format Page Descriptor and Short-Format Early Termination Page Descriptor**

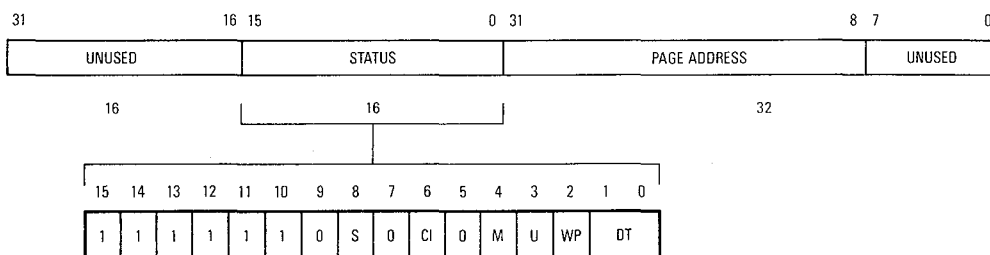
**9.5.1.6 LONG-FORMAT EARLY TERMINATION PAGE DESCRIPTOR.** The long-format early termination page descriptor contains the page descriptor code in the DT field but resides in a pointer table like the short-format early termination page descriptor. The field descriptions in **9.5.1.1 DESCRIPTOR FIELD DEFINITIONS** apply to corresponding fields of this descriptor. Figure 9-13 shows the format of the long-format early termination page descriptor. The LIMIT field of the long-format descriptor provides a means of limiting the number of pages to which the descriptor applies.



**Figure 9-13. Long-Format Early Termination Page Descriptor**

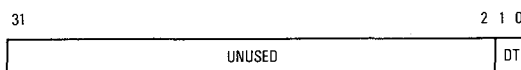
**9.5.1.7 SHORT-FORMAT PAGE DESCRIPTOR.** The short-format page descriptor is used in the page tables (the bottom level of the address table). The field descriptions in **9.5.1.1 DESCRIPTOR FIELD DEFINITIONS** apply to the corresponding fields of this descriptor. The short-format page descriptor is identical to of the short-format early termination page descriptor shown in Figure 9-12.

**9.5.1.8 LONG-FORMAT PAGE DESCRIPTOR.** The long-format page descriptor is also used in the page tables. The field descriptions in **9.5.1.1 DESCRIPTOR FIELD DEFINITIONS** apply to the corresponding fields of this descriptor. Figure 9-14 shows the format of the long-format page descriptor.



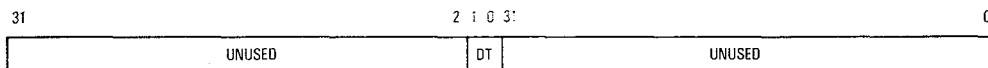
**Figure 9-14. Long-Format Page Descriptor**

**9.5.1.9 SHORT-FORMAT INVALID DESCRIPTOR.** The short-format invalid descriptor consists of a DT field that contains zeros, identifying it as an invalid descriptor. It can be used at any level of the address translation tree except at the root pointer level. The 30-bit unused field is available to the operating system to identify unallocated portions of the table or portions of the table that reside on an external device. For example, the disk address of disk-resident tables or pages can be stored in this field. Figure 9-15 shows the format of a short-format invalid descriptor.



**Figure 9-15. Short-Format Invalid Descriptor**

**9.5.1.10 LONG-FORMAT INVALID DESCRIPTOR.** The long-format invalid descriptor is used in pointer and page tables that contain long-format descriptors. It is used in the same way as the short-format invalid descriptor in the preceding section. The first long word contains the DT field in the lowest order bits. The second long word is an unused field, also available to the operating system. Figure 9-16 shows the format of the long-format invalid descriptor.



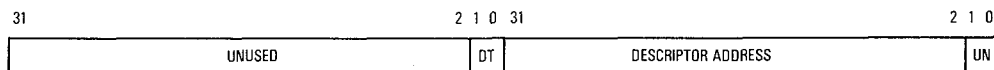
**Figure 9-16. Long-Format Invalid Descriptor**

**9.5.1.11 SHORT-FORMAT INDIRECT DESCRIPTOR.** The short-format indirect descriptor does not have a unique descriptor-type code. Rather, it resides in a page table (the bottom level of the address translation tree) that contains short-format descriptors and is neither a page descriptor nor an invalid descriptor. The descriptor-type field contains either the code for a valid 4-byte descriptor or for a valid 8-byte descriptor, depending upon the size of the referenced page descriptor. The field descriptions in **9.5.1.1 DESCRIPTOR FIELD DEFINITIONS** apply to the corresponding fields of this descriptor. Figure 9-17 shows the format of a short-format indirect descriptor.



**Figure 9-17. Short-Format Indirect Descriptor**

**9.5.1.12 LONG-FORMAT INDIRECT DESCRIPTOR.** The long-format indirect descriptor has all the attributes of the short-format indirect descriptor described in the preceding section. The only differences are that it is used in a page table that contains long-format descriptors and that it has two unused fields. The field descriptions in **9.5.1.1 DESCRIPTOR FIELD DEFINITIONS** apply to corresponding fields of this descriptor. Figure 9-18 shows the format of a long-format indirect descriptor.



**Figure 9-18. Long-Format Indirect Descriptor**

## 9.5.2 General Table Search

When the ATC does not contain a descriptor for the logical address of a processor access and a translation is required, the MC68030 searches the translation tables in memory and obtains the physical address and status information for the page corresponding to the logical address. When a table search is required, the CPU suspends instruction execution activity and, at the end of a successful table search, stores the address mapping in the ATC and retries the access. The access then results in a match (it hits) and the translated address is transferred to the bus controller (provided no exceptions were encountered).

The table search begins by selecting the translation tree, using function code bit FC2 and the SRE bit of the TC register, as shown in Table 9-2. SRE is set to enable the supervisor root pointer, and FC2 is set for supervisor-level accesses. The translation tree with its root defined by the SRP register is selected only when SRE and FC2 are both set. Otherwise, the translation table with its root defined by the CRP register is selected. A simplified flowchart of the table search procedure is shown in Figure 9-19.

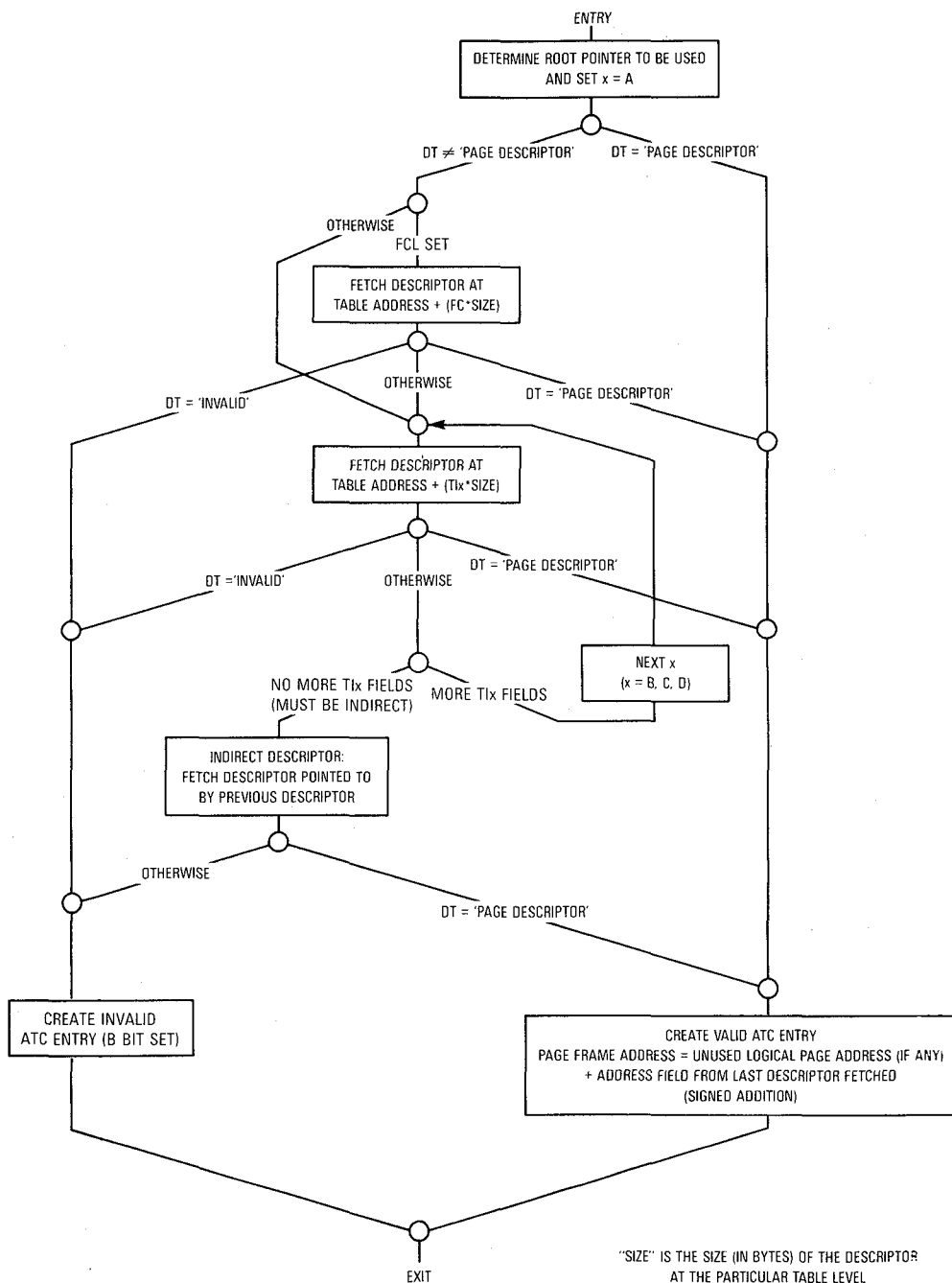


Figure 9-19. Simplified Table Search Flowchart

**Table 9-2. Translation Tree Selection**

| FC2 | SRE | Translation Table Root Pointer |
|-----|-----|--------------------------------|
| 0   | 0   | CRP                            |
| 0   | 1   | CRP                            |
| 1   | 0   | CRP                            |
| 1   | 1   | SRP                            |

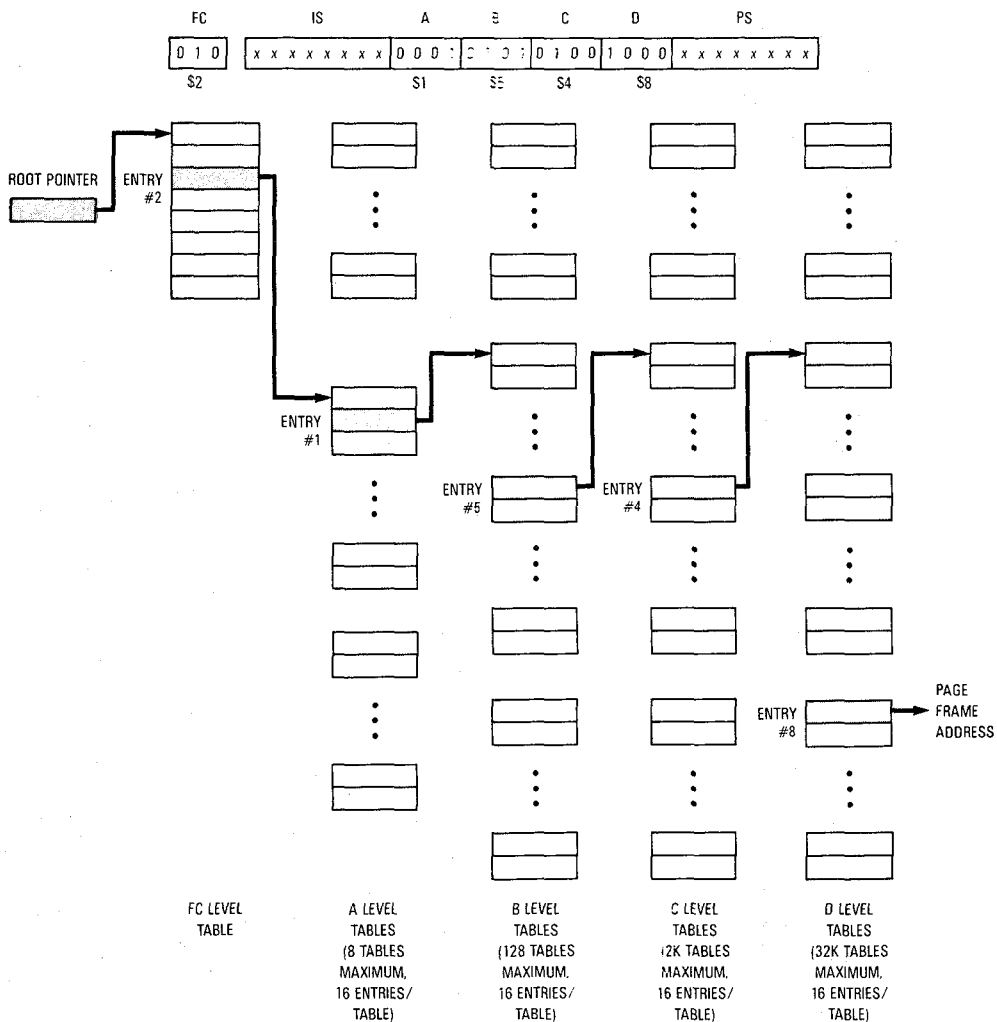
The table search procedure uses physical addresses to access the translation tables. The read-modify-write (RMC) signal is asserted on the first bus cycle of the search and remains asserted throughout, ensuring that the entire table search completes without interruption.

The first bus cycle of the search uses the table address field of the appropriate root pointer as the base address of the first table. The low-order bits of the address are supplied by the logical address. The table is indexed by either the function code or the set of logical address bits defined by the TIA field of the TC register. The FCL field of the TC register determines whether or not the function code is used. In either case, the descriptor-type field of the root pointer selects the scale factor (or multiplier) for the index.

The first access obtains a descriptor. If the descriptor is a table descriptor, the MC68030 again accesses memory. The next access uses the table address in the descriptor as the base address for the next table. The low-order bits of the address are supplied by the logical address. The table is indexed by a set of bits from the logical address using a scale factor determined by the descriptor type code in the descriptor. If the first table access used the function code, the second access uses the bits selected by the TIA field of the TC register. Otherwise, the second access uses the bits selected by the TIB field.

Additional accesses are performed, using the logical address bits specified in TIB, TIC, or TID in order, until an access obtains a page descriptor or an invalid descriptor or until a limit violation occurs. At this point, whether or not all levels of the address table have been accessed, the table search is over. The page descriptor contains the physical address and other information needed for the ATC entry; the MC68030 creates the ATC entry and retries the original bus access.

Figure 9-20 shows a table search using the function code and all four Tlx fields.



**Figure 9-20. Five-Level Table Search**

The MC68030 enforces a limit on the index value for the next level of a table search when long-format descriptors are used.

The root pointer includes a limit field that applies when the function code lookup is not used (the FCL bit of the TC register is zero). The index used to access the next level table is compared to the contents of the limit field. The limit field effectively reduces the portion of the address space to which a descriptor applies and also reduces the size of the translation table. The index must reside within the range defined by the limit field. The limit can be a lower limit or an upper limit, according to the L/U bit value. When the L/U bit is set, the limit is a lower limit, and an index less than the limit is out of bounds. When the L/U bit is zero, the limit is an upper limit, and an index greater than the limit is out of bounds. The limit field is effectively disabled if L/U is set and the limit field contains zero or if L/U is clear and the limit field contains \$7FFF.

During a table search for an normal translation or a PLOAD instruction, if a limit violation is detected, the ATC is loaded with an entry having the bus error (B) bit set. If a limit violation is detected during a table search for a PTEST instruction, the invalid (I) and limit (L) bits are set in the MMUSR.

During a table search, the U bit in each descriptor that is encountered is checked and set if it is not already set. Similarly, when the table search is for a write access and the M bit of the page descriptor is clear, the processor sets the bit if the table search does not encounter a set WP bit or a supervisor violation. Since the read-modify-write (RMC) signal is asserted throughout the entire table search operation, the read and write operations to update the history bits are guaranteed to be uninterrupted.

A table search terminates successfully when a page descriptor is encountered. The occurrence of an invalid descriptor, a limit violation, or a bus error also terminates a table search, and the MC68030 takes an exception on the retry of the cycle because of these conditions. The exception routine should distinguish between anticipated conditions and true error conditions. The routine can correct an invalid descriptor that indicates a nonresident page or one that identifies a portion of the translation table yet to be allocated. A limit violation or a bus error due to a system malfunction may result in an error message and termination of the task.

## 9.5.3 Variations in Translation Table Structure

Many aspects of the MMU translation tree structure are software configurable, allowing the system designer flexibility to optimize the performance of the MMU for a particular system. The following paragraphs discuss the variations of the tree structure from the general structure discussed previously.

**9.5.3.1 EARLY TERMINATION AND CONTIGUOUS MEMORY.** The MMU provides the ability to map a contiguous range of the logical address space (an integral number of logical pages) to an equivalent contiguous physical address range with a single descriptor. This is done by placing the code for page descriptor (\$1) in the descriptor type (DT) field of a descriptor at a level of the tree that would normally contain a table pointer, thereby deleting a subtree of the table.

The table search ends when the search encounters a page descriptor, whether or not the page descriptor is in a page descriptor table at the lowest level of the translation tree.

Termination of the table search by a page descriptor in a pointer descriptor table (i.e., the MC68030 has not encountered a Tlx field of zero) is called an early termination. The terminating page descriptor is called an early termination page descriptor.

An early termination page descriptor takes the place of many page descriptors in a translation table. It applies to all pages that would exist on the branch on which the descriptor has been placed, and on any branches from that branch. An early termination page descriptor can be used where contiguous pages in physical memory correspond to contiguous logical pages. If an early termination page descriptor is a long format, the limit field is applied to the next index field of the logical address. This allows the number of pages mapped contiguously to be restricted. Refer to **9.1.2 Translation Table Descriptors** for additional information.

If  $n$  low-order bits of the logical page address are unused when a page descriptor encoding is encountered, the single descriptor creates a mapping of a contiguous region of the logical address space starting at the logical page address (with  $n$  unused bits set to zero) to a contiguous region in the physical address space starting at the page frame base address with a size of  $2^{PS+n}$  bytes.

When a search is made for a logical address to which an early termination page descriptor applies, the MC68030 creates an entry in the ATC for the logical address; the physical address in the ATC entry is the sum of the page address field in the descriptor plus an offset. The offset is the logical address with the bits used in the search set to zero.

Although the early termination page descriptor creates a contiguous logical-to-physical mapping without having to maintain individual descriptors in the translation tree for each page that is a member of the contiguous region, the ATC contains one entry for each page mapped. These entries are created internally each time a page boundary (as determined by the page size) is crossed in the contiguous region. Figure 9-21 shows an example translation table with a portion of the logical address space translated as a contiguous block.

Note that the DT field can be set to page descriptor at any level of the translation tree including the root pointer level. Setting the DT field of a root pointer to page descriptor creates a direct mapping from the logical to the physical address space with a constant offset as determined by the value in the table address field of the root pointer.

**9.5.3.2 INDIRECTION.** The MC68030 provides the ability to replace an entry in a page table with a pointer to an alternate entry. The indirection capability allows multiple tasks to share a physical page while maintaining only a single set of history information for the page (i.e., the “modified” indication is maintained only in the single descriptor). The indirection capability also allows the page frame to appear at arbitrarily different addresses in the logical address spaces of each task.

Using the indirection capability, single entries or entire tables can be shared between multiple tasks. Figure 9-22 shows two tasks sharing a page using indirect descriptors.

When the MC68030 has completed a normal table search (has exhausted all index fields of the logical page address), it examines the descriptor-type field of the last entry fetched from the translation tables. If the DT field contains a “valid long” (\$2) or “valid short” (\$3) encoding, this indicates that the address contained in the highest order 30 bits of the table address field of the descriptor is a pointer to the page descriptor that is to be used to map the logical address. The processor then fetches the page descriptor of the indicated format from this address and uses the page address field of the page descriptor as the physical mapping for the logical address.

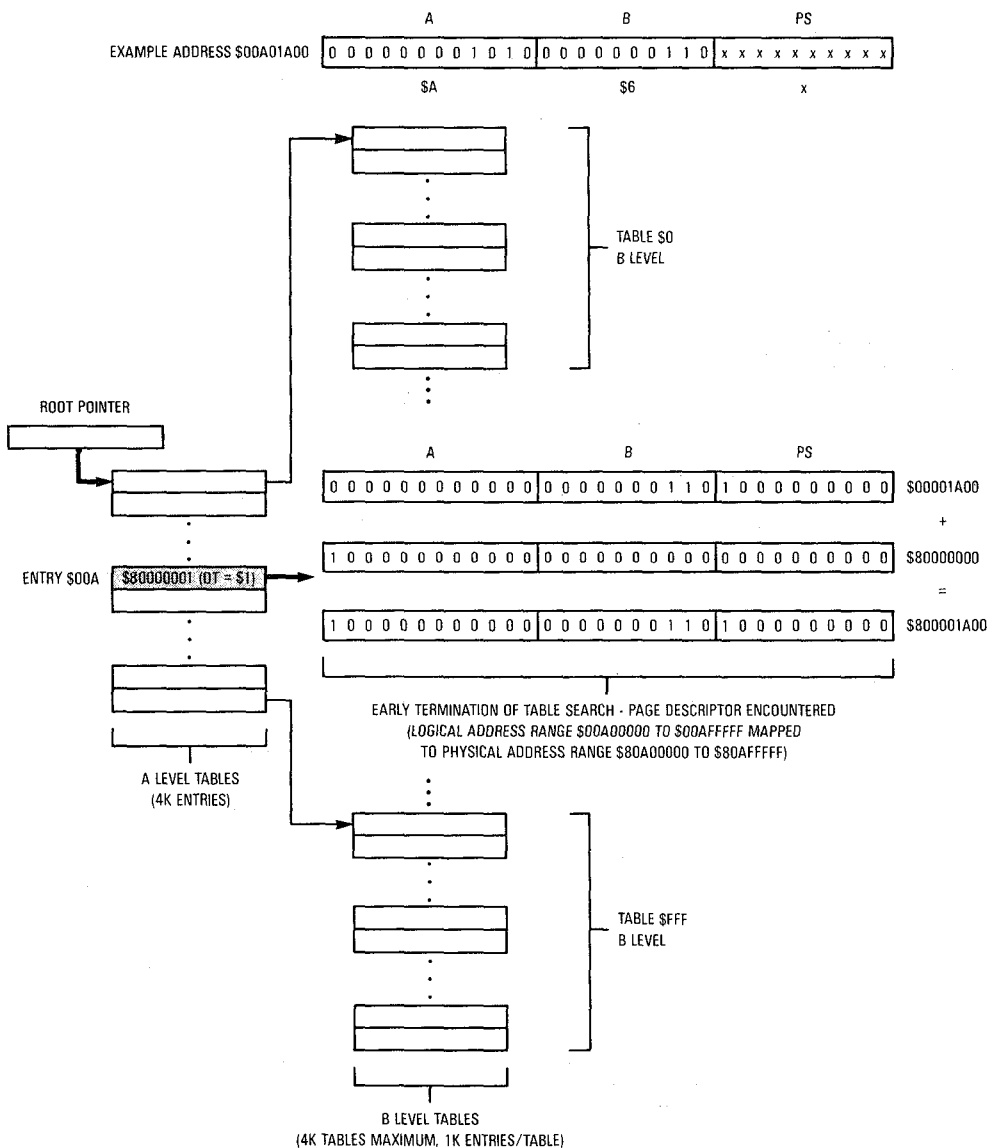


Figure 9-21. Example Translation Tree Using Contiguous Memory

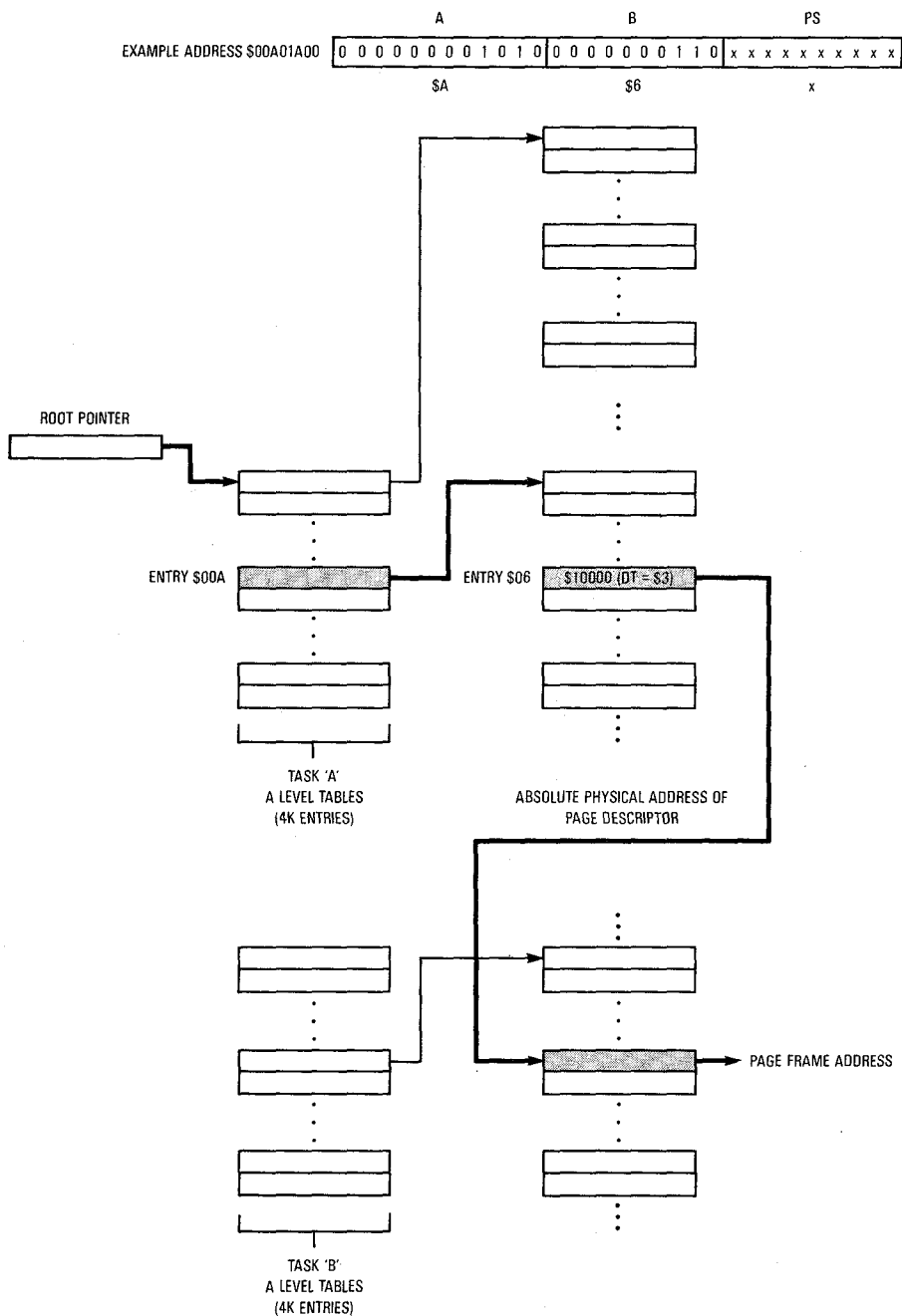


Figure 9-22. Example Translation Tree Using Indirect Descriptors

The page descriptor located at the address given by the indirect descriptor must not have a DT field with a long or short encoding (it must either be a page descriptor or invalid). Otherwise, the descriptor is treated as invalid, and the MC68030 creates an ATC entry with an error condition signaled (bit set).

**9.5.3.3 TABLE SHARING BETWEEN TASKS.** A page or pointer table can be shared between tasks by placing a pointer to the shared table in the address translation tables of more than one task. The upper (nonshared) tables can contain different settings of protection bits allowing different tasks to use the area with different permissions. In Figure 9-23 two tasks share the memory translated by the table at the B level. Note that task "A" cannot write to the shaded area. Task "B", however, has the WP bit clear in its pointer to the shared table; thus, it can read and write the shared area. Also note that the shared area appears at different logical addresses for each task.

**9.5.3.4 PAGING OF TABLES.** It is not required that the entire address translation tree for an active task be resident in main memory at once. In the same way that only the working set of pages must reside in main memory, only the tables that describe the resident set of pages need be available in main memory. This paging of tables is implemented by placing the "invalid" code (\$0) in the DT field of the table descriptor that points to the absent table(s). When a task attempts to use an address that would be translated by an absent table, the MC68030 is unable to locate a translation and takes a bus error exception when the execution unit retries the bus cycle that caused the table search to be initiated.

It is the responsibility of the system software to determine that the invalid code in the descriptor corresponds to nonresident tables. This determination can be facilitated by using the unused bits in the descriptor to store status information concerning the invalid encoding. When the MC68030 encounters an invalid descriptor, it makes no interpretation (or modification) of any fields of this descriptor other than the DT field, allowing the operating system to store system-defined information in the remaining bits. Typical information that is stored includes the reason for the invalid encoding (tables paged-out, region not allocated, . . . , etc.) and possibly the disk address for nonresident tables.

Figure 9-24 shows an address translation table in which only a single page table (table n) is resident and all other page tables are not resident.

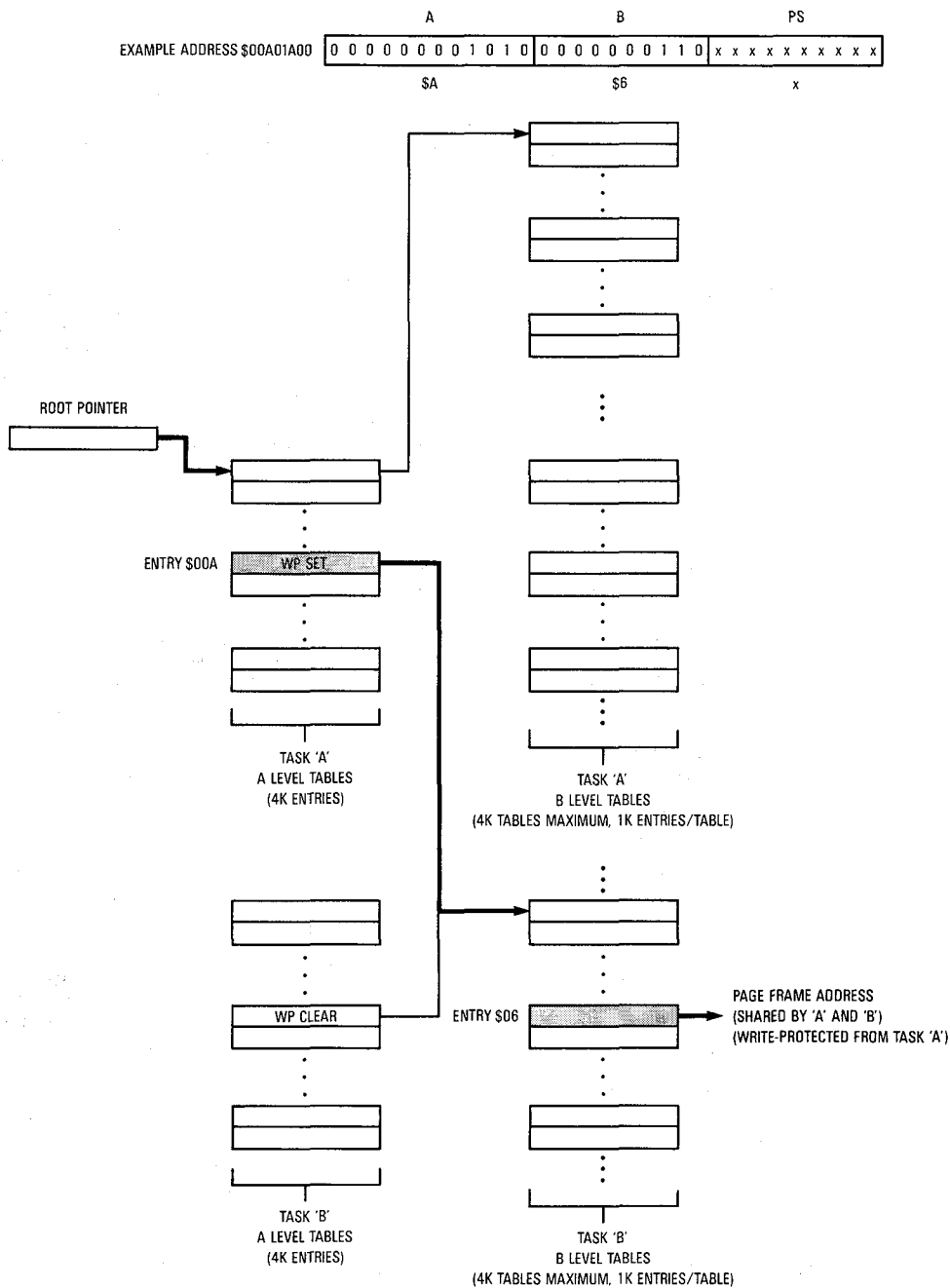


Figure 9-23. Example Translation Tree Using Shared Tables

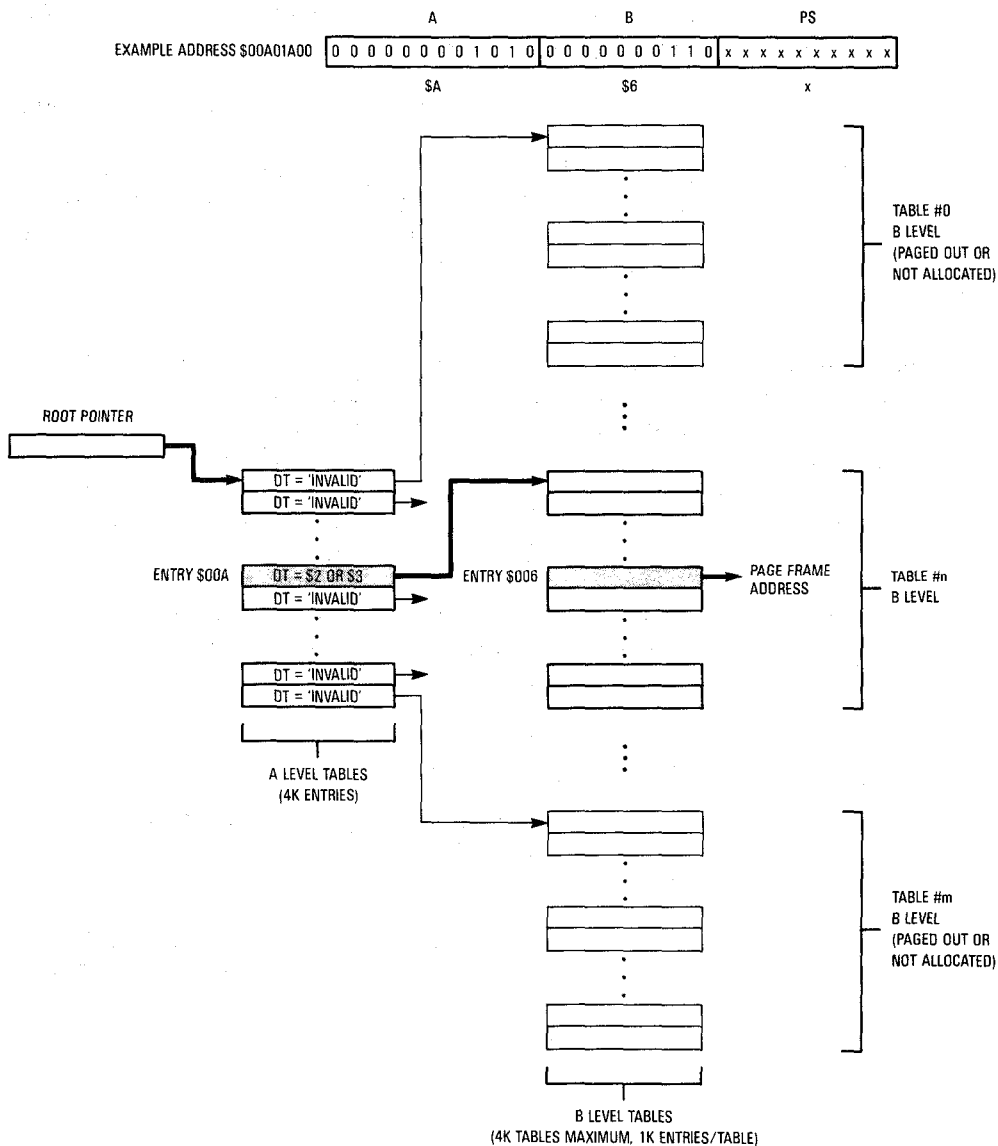


Figure 9-24. Example Translation Tree with Nonresident Tables

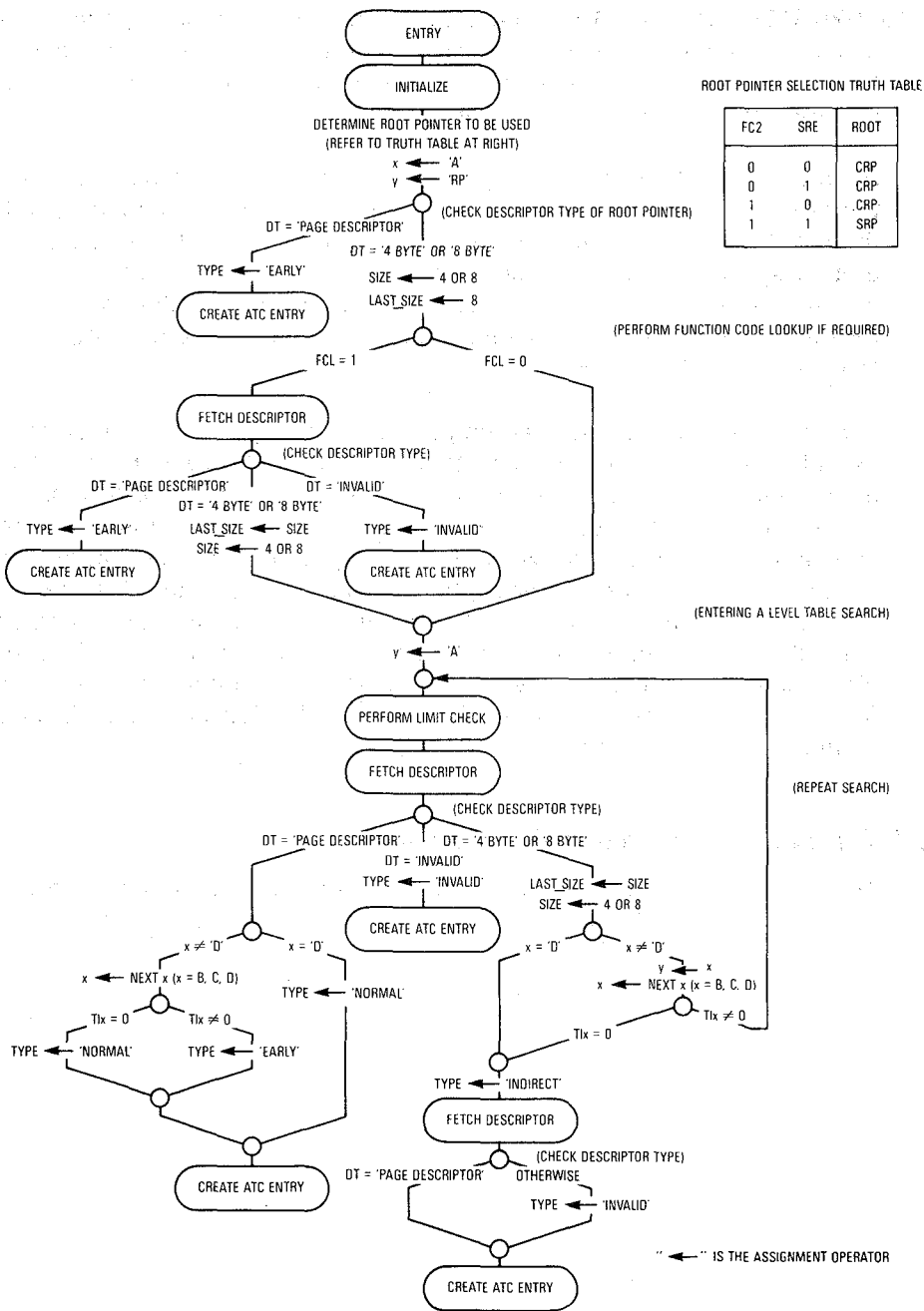
**9.5.3.5 DYNAMIC ALLOCATION OF TABLES.** Similar to the case of paged tables, it is not required that a complete translation tree exist for an active task. The translation tree can be dynamically allocated by the operating system based on requests for access to particular areas.

As in the case of demand paging, it is difficult, if not impossible, to predict the areas of memory that are used by a task over any extended period of time. Instead of attempting to predict the requirements of the task, the operating system performs no action for a task until a demand is made requesting access to a previously unused area or an area that is no longer resident in memory. This same technique can be used to efficiently create a translation tree for a task.

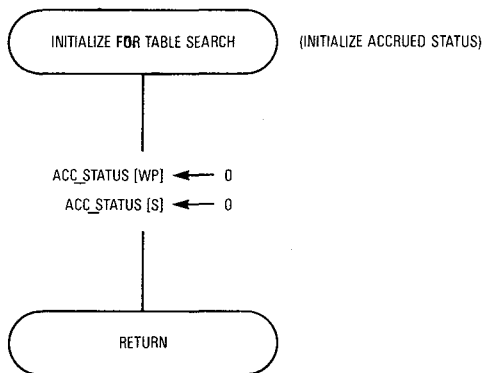
For example, consider an operating system that is preparing the system to execute a previously unexecuted task that has no translation tree. Rather than guessing what the memory usage requirements of the task are, the operating system creates a translation tree for the task that maps one page corresponding to the initial value of the program counter for that task, and possibly, one page corresponding to the initial stack pointer of the task. All other branches of the translation tree for this task remain unallocated until the task requests access to the areas mapped by these branches. This technique allows the operating system to construct a minimal translation tree for each task, conserving physical memory utilization and minimizing operating system overhead.

#### 9.5.4 Detail of Table Search Operations

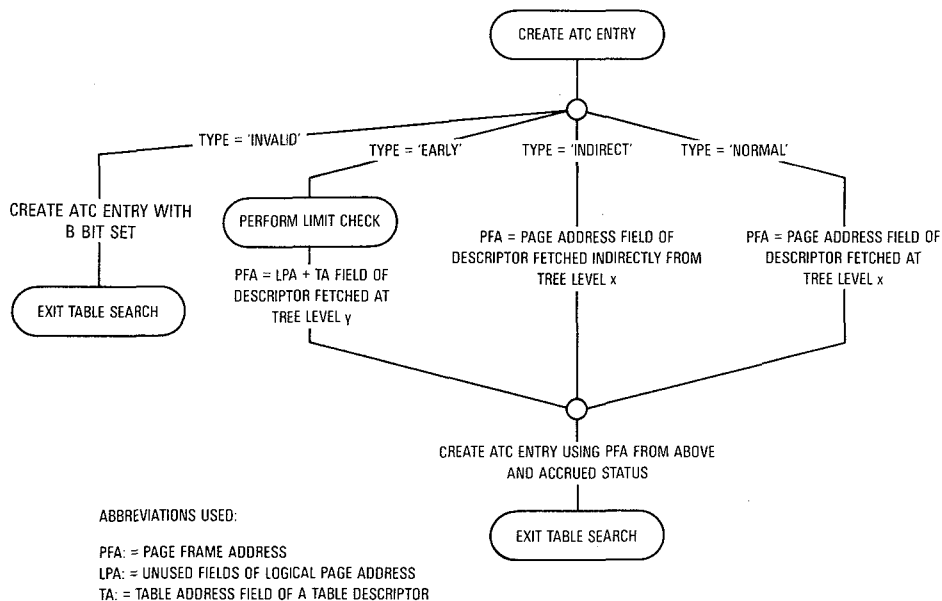
The table search operations described in this section are shown in detail in Figures 9-25–9-29.



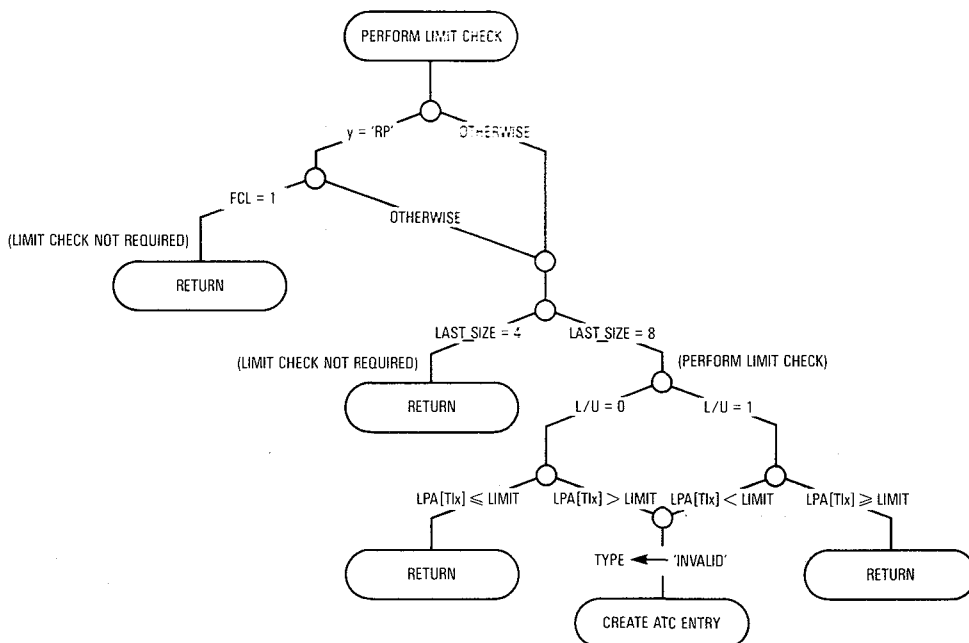
**Figure 9-25. Detailed Flowchart of MMU Table Search Operation**



**Figure 9-26. Table Search Initialization Flowchart**



**Figure 9-27. ATC Entry Creation Flowchart**



**Figure 9-28. Limit Check Procedure Flowchart**

### 9.5.5 Protection

9

M68000 Family processors provide an indication of the context in which they are operating on a cycle-by-cycle basis by means of the function code signals. These signals identify accesses to the user program space, the user data space, the supervisor program space, and the supervisor data space. The function code signals can be used for protection mechanisms by setting the function code lookup (FCL) bit in the translation control (TC) register.

The MC68030 MMU provides the capability for separate translation trees for supervisor and user spaces to be used. When the supervisor root pointer enable bit (SRE) in the TC register is set, the root pointer register for the supervisor space translation tree is selected for supervisor program or data accesses.

The translation table trees contain both mapping and protection information. Each table and page descriptor includes a write-protect (WP) bit, which can be set to provide write protection at any level. Each long-format table and page descriptor also contains a supervisor-only (S) bit, which can limit access to programs operating at the supervisor privilege level.

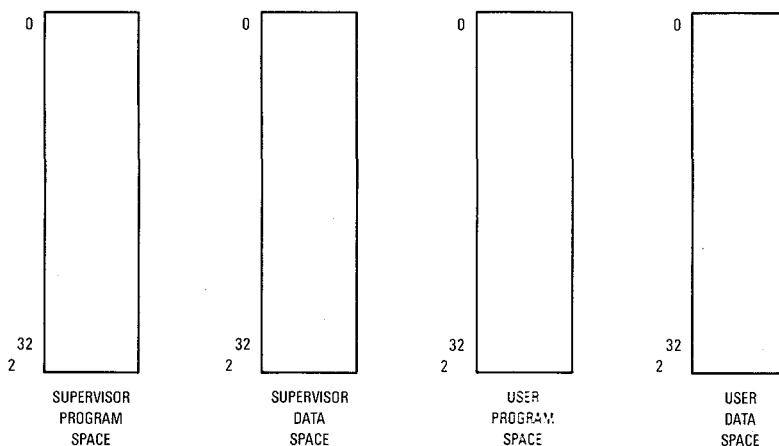


9-44

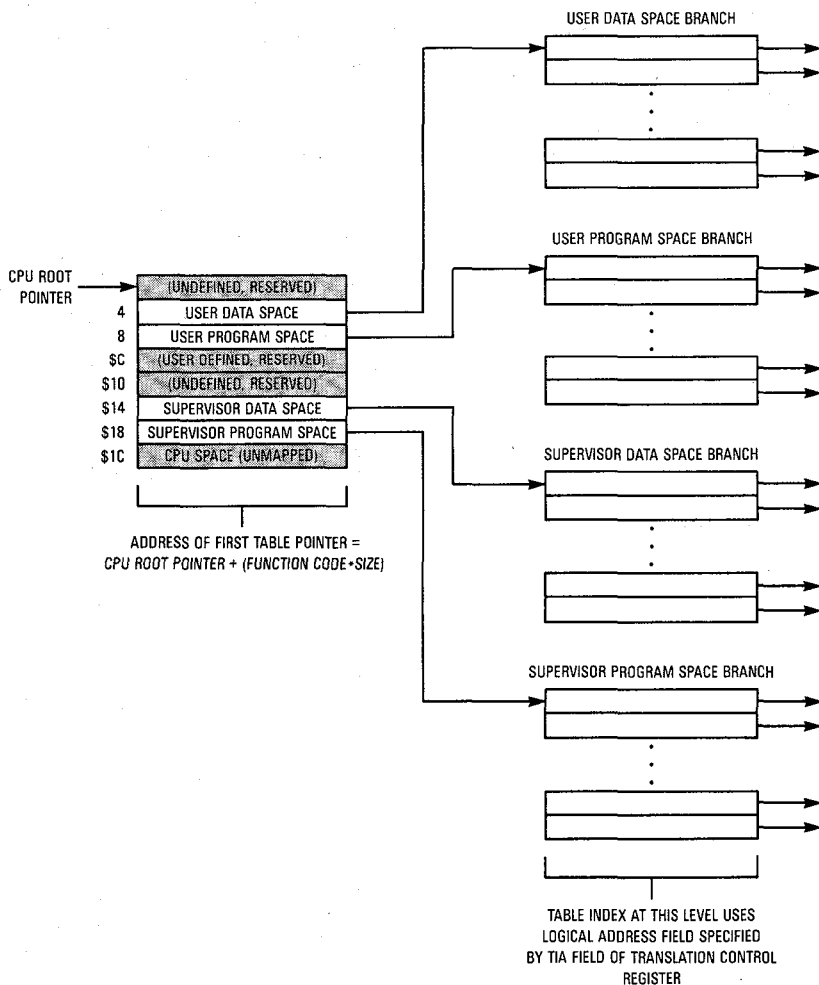
The protection mechanisms can be used individually or in any combination to protect:

- Supervisor program and data spaces from access by user programs.
- User program and data spaces from access by other user programs or supervisor programs (except with the MOVES instruction).
- Supervisor and user program spaces from write accesses (except by the supervisor using the MOVES instruction).
- One or more pages of memory from write accesses.

**9.5.5.1 FUNCTION CODE LOOKUP.** One way of protecting supervisor and user spaces from unauthorized access is to set the FCL bit in the TC register. This effectively segments the logical address space into a supervisor program space, a supervisor data space, a user program space, and a user data space, as shown in Figure 9-30. Each task has an address translation tree with unique mappings for the logical addresses in its user spaces. The translation tables for mapping the supervisor spaces can be copied into each task's translation tree. Figure 9-31 shows a translation tree using function code lookup, and Figure 9-32 shows translation trees for two tasks that share common supervisor spaces.



**Figure 9-30. Logical Address Map Using Function Code Lookup**



**Figure 9-31. Example Translation Tree Using Function Code Lookup**

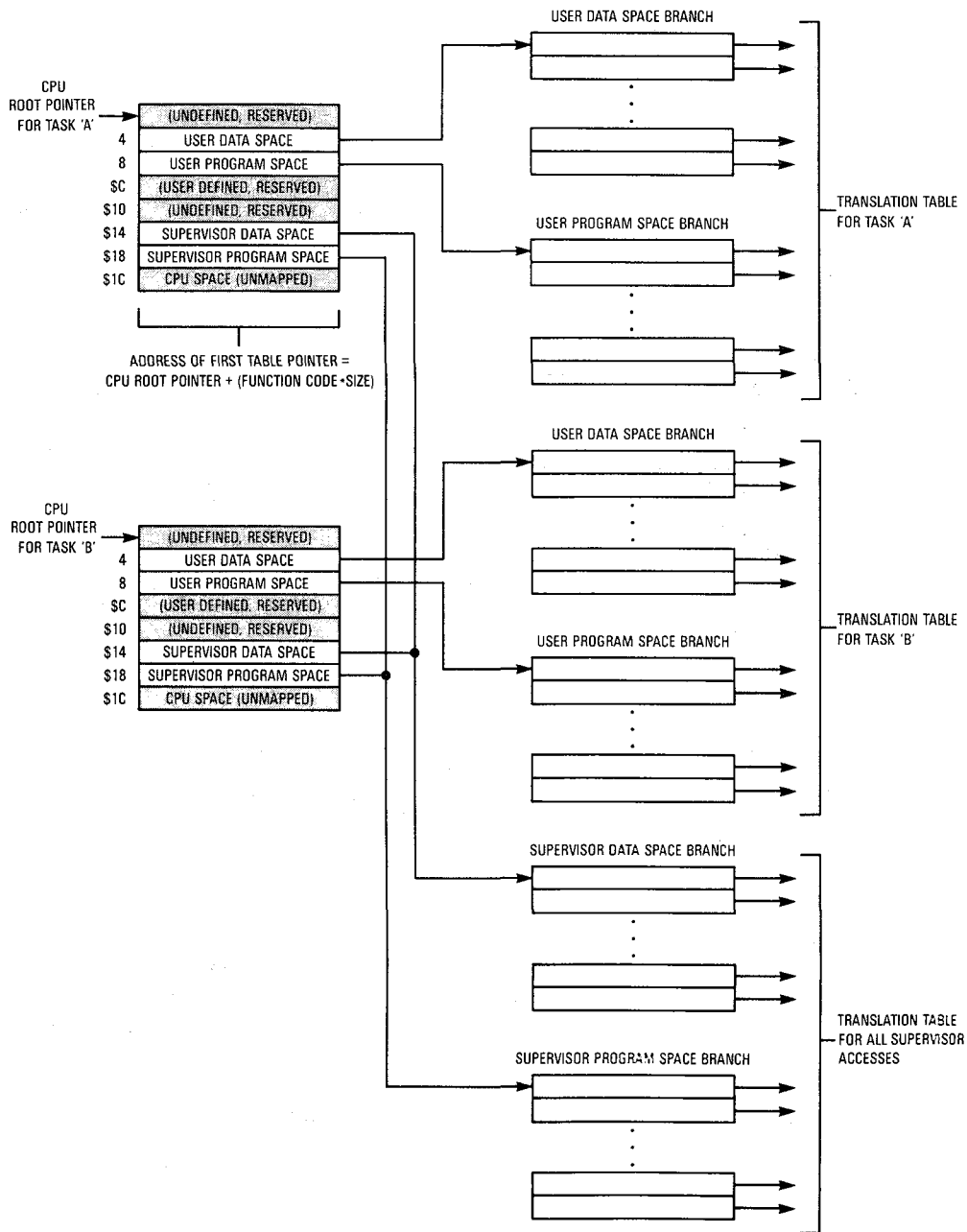
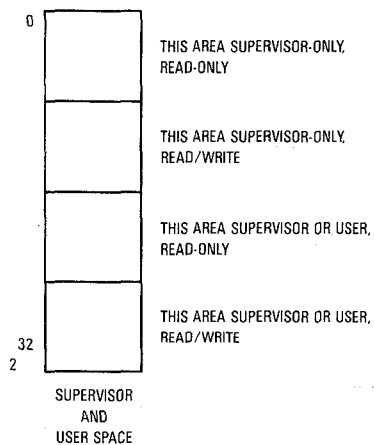


Figure 9-32. Example Translation Tree Structure for Two Tasks

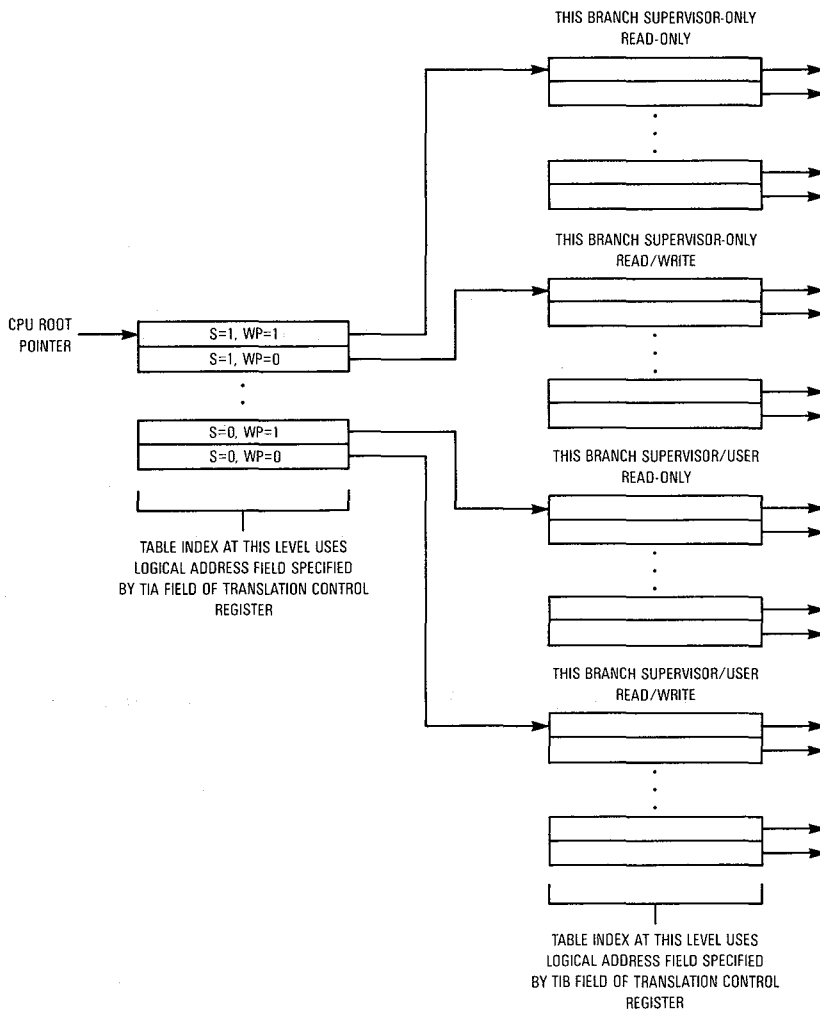
**9.5.5.2 SUPERVISOR TRANSLATION TREE.** A second protection mechanism uses a supervisor translation tree. A supervisor translation tree protects supervisor programs and data from access by user programs and user programs and data from access by supervisor programs. Access is granted to the supervisor programs which can access any area of memory with the move address space (MOVES) instruction. When the SRE bit in the TC register is set, the translation tree pointed to by the SRP is selected for all supervisor level accesses. This translation tree can be common to all tasks. This technique segments the logical address space into user and supervisor areas without adding the function code level to the translation trees.

**9.5.5.3 SUPERVISOR ONLY.** A third mechanism protects supervisor programs and data without segmenting the logical address space into supervisor and user address spaces. The long formats of table descriptors and page descriptors contain S bits to protect areas of memory from access by user programs. When a table search for a user access encounters an S bit set in any table or page descriptor, the table search is completed and an ATC descriptor corresponding to the logical address is created with the B bit set. The subsequent retry of the user access results in a bus error exception being taken. The S bit can be used to protect the entire area of memory defined in a branch of the translation tree or only one or more pages from user program access.

**9.5.5.4 WRITE PROTECT.** The MC68030 provides write protection independently of the segmented address spaces for programs and data. All table and page descriptors contain WP bits to protect areas of memory from write accesses of any kind. When a table search encounters a WP bit set in any table or page descriptor, the table search is completed and an ATC descriptor corresponding to the logical address is created with the WP bit set. The subsequent retry of the write access results in a bus error exception being taken. The WP bit can be used to protect the entire area of memory defined in a branch of the translation tree, or only one or more pages from write accesses. Figure 9-33 shows a memory map of the logical address space organized to use S and WP bits for protection. Figure 9-34 shows an example translation tree for this technique.



**Figure 9-33. Example Logical Address Map with Shared Supervisor and User Address Spaces**



**Figure 9-34. Example Translation Tree Using S and WP Bits to Set Protection**



## 9.6 MC68030 AND MC68851 MMU DIFFERENCES

The MC68851 paged memory management unit provides memory management for the MC68020 as a coprocessor. The on-chip MMU of the MC68030 provides many of the features of the MC68020/MC68851 combination. The following functions of the MC68851 are not available in the MC68030 MMU:

- Access Levels
- Breakpoint Registers
- Root Pointer Table
- Aliases for Tasks
- Lockable Entries in the ATC
- ATC Entries Defined as Shared Globally

In addition, the following features of the MC68030 MMU differ from the MC68020/MC68851 pair:

- 22-Entry ATC
- Reduced Instruction Set
- Only Control-Alterable Addressing Modes Supported for MMU Instructions

In general, the MC68030 is program compatible with the MC68020/MC68851 combination. However, in a program for the MC68030, the following instructions must be avoided or emulated in the exception routine for F-line unimplemented instructions: PVALID, PFLUSHR, PFLUSHS, PBcc, PDBcc, PScC, PTRAPcc, PSAVE, PRESTORE, and PMOVE for unsupported registers (CAL, VAL, SCC, BAD, BACx, DRP, and AC). Additionally, the effective addressing modes supported on the MC68851 that are not emulated by the MC68030 must be simulated or avoided.

## 9.7 REGISTERS

The registers of the MMU described here are part of the supervisor programming model for the MC68030.

The six registers that control and provide status information for address translation in the MC68030 are the CPU root pointer register (CRP), the supervisor root pointer register (SRP), the translation control register (TC), two independent transparent translation control registers (TT0 and TT1), and the MMU status register (MMUSR). These registers can be accessed directly by programs that execute only at the supervisor level.

### 9.7.1 Root Pointer Registers

The supervisor root pointer (SRP), used for supervisor accesses only, is enabled or disabled in software. The CPU root pointer (CRP) corresponds to the current translation table for user space (when the SRP is enabled) or for both user and supervisor space (when the SRP is disabled). The CRP is a 64-bit register that contains the address and related status information of the root of the translation table tree for the current task. When a new task begins execution, the operating system typically writes a new root pointer descriptor to the CRP. A new translation table address implies that the contents of the address translation cache (ATC) may no longer be valid. Therefore, the instruction that loads the CRP can optionally flush the ATC.

The SRP is a 64-bit register that optionally contains the address and related status information of the root of the translation table for supervisor area accesses. The SRP is used when operating at the supervisor privilege level only when the supervisor root pointer enable bit (SRE) of the translation control register (TC) is set. The instruction that loads the SRP can optionally flush the ATC. The format of the CRP and SRP is shown in Figure 9-35 and defines the following fields:

#### Lower/Upper (L/U)

Specifies that the value contained in the limit field is to be used as the unsigned lower limit of indexes into the translation tables when this bit is set. When this bit is cleared, the limit field is the unsigned upper limit of the translation table indexes.

## Limit

Specifies a maximum or minimum value for the index to be used at the next level of table search (the function code level cannot be limited). To suppress the limit function, the L/U bit is cleared and the limit field is set to ones (\$7FFF in the word containing both fields), or the L/U bit is set and the limit field is cleared (\$8000 in that word).

## Descriptor Type (DT)

Specifies the type of descriptor contained in either the root pointer or in the first level of the translation table identified by the root pointer. The values are:

### \$0 INVALID

This value is not allowed at the root pointer level. When a root pointer register is loaded with an invalid root pointer descriptor, an MMU configuration exception is taken.

### \$1 PAGE DESCRIPTOR

A translation table for this root pointer does not exist. The MC68030 internally calculates an ATC entry (page descriptor) for accesses using this root pointer within the current page by adding (unsigned) the value in the table address field to the incoming logical address. This results in direct mapping with a constant offset (the table address). For this case, the processor performs a limit check, regardless of the state of the FCL bit in the TC register.

### \$2 VALID 4 BYTE

The translation table at the root of the translation tree contains short-format descriptors. The MC68030 must scale the table index for this level of the table search by 4 bytes to access the next descriptor.

### \$3 VALID 8 BYTE

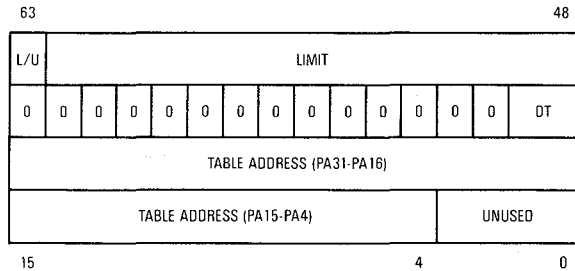
The translation table at the root of the translation tree contains long-format descriptors. The MC68030 must scale the table index for this level of the table search by 8 bytes to access the next descriptor.

## Table Address

Contains the physical base address (in bits 31–4) of the translation table at the root pointer level. When the DT field contains \$1, the value in the table address field is the offset used to calculate the physical address for the page descriptor. The table address field can contain zero (for zero offset).

## Unused

Bits 3–0 of the root pointer are not used and are ignored when written. All other unused bits must always be zeros.



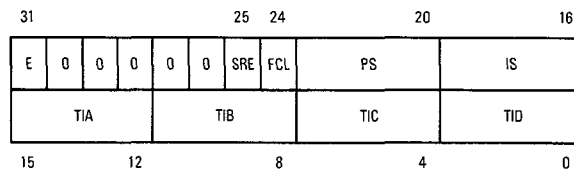
L/U — LOWER OR UPPER PAGE RANGE  
 DT — DESCRIPTOR TYPE  
 LIMIT — LIMIT ON TABLE INDEX FOR THIS TABLE ADDRESS  
 TABLE ADDRESS — ADDRESS OF TABLE AT NEXT LEVEL OR PAGE OFFSET IF DT = 1

**Figure 9-35. Root Pointer Register (CRP, SRP) Format**

## 9.7.2 Translation Control Register

The translation control register (TC) is a 32-bit register that contains the control fields for address translation. All unimplemented fields of this register are read as zeros and must always be written as zeros.

Writing to this register optionally causes a flush of the entire ATC. When written with the E bit (bit 31) set (translation enabled), a consistency check is performed on the values of PS, IS, and Tlx as follows. The Tlx fields are added together until a zero field is reached, and this sum is added to PS and IS. The total must be 32, or an MMU configuration exception (refer to **9.7.5.3 MMU CONFIGURATON EXCEPTION**) is taken. If an MMU configuration exception occurs, the TC register is updated with the data, and the E bit is cleared. The translation control register is shown in Figure 9-36.



E — ENABLE  
 SRE — SUPERVISOR ROOT POINTER ENABLE  
 FCL — FUNCTION CODE LOOKUP ENABLE  
 PS — PAGE SIZE  
 IS — INITIAL SHIFT  
 TIA, TIB, TIC, TID — TABLE INDICES

**Figure 9-36. Translation Control Register (TC) Format**

The fields of the TC register are:

#### Enable (E)

This bit enables and disables address translation:

- 0 — Translation disabled
- 1 — Translation enabled

A reset operation clears this bit. When translation is disabled, logical addresses are used as physical addresses. The MMU instructions (PTEST, PLOAD, PMOVE, PFLUSH) can be executed successfully, regardless of the state of the E bit. Additionally, even if the E bit is set, the TC register can be updated with a value whose E bit is set. The state of the E bit does not affect the use of the transparent translation registers.

#### Supervisor Root Pointer Enable (SRE)

This bit controls the use of the supervisor root pointer register (SRP):

- 0 — SRP disabled
- 1 — SRP enabled

When the SRP is disabled, both user and supervisor accesses use the translation table defined by the CRP. When the SRP is enabled, user accesses use the CRP, and supervisor accesses use the SRP.

#### Function Code Lookup (FCL)

This bit enables the use of function code lookup for searching the address translation tables:

- 0 — Function code lookup disabled
- 1 — Function code lookup enabled

When function code lookup is disabled, the first level of pointer tables within the translation table structure is indexed by the logical address field defined by TIA. When function code lookup is enabled, the first table of the translation table structure is indexed by function code. In this case, the limit field of CRP or SRP is ignored.

## Page Size (PS)

This 4-bit field specifies the system page size:

1000 — 256 bytes

1001 — 512 bytes

1010 — 1K bytes

1011 — 2K bytes

1100 — 4K bytes

1101 — 8K bytes

1110 — 16K bytes

1111 — 32K bytes

All other bit combinations are reserved by Motorola for future use; an attempt to load other values into this field of the TC register causes an MMU configuration exception.

## Initial Shift (IS)

This 4-bit field contains the number of high-order bits of the logical address that are ignored during table search operations. The field contains an integer, 0–15, which sets the effective size of the logical address to 32–17 bits, respectively. Since all 32 bits of the address are compared during address translation, bits ignored due to initial shift cannot have random values. They must be specified and be consistent with the translation table values in order to ensure that subsequent address translations match the corresponding entries in the ATC.

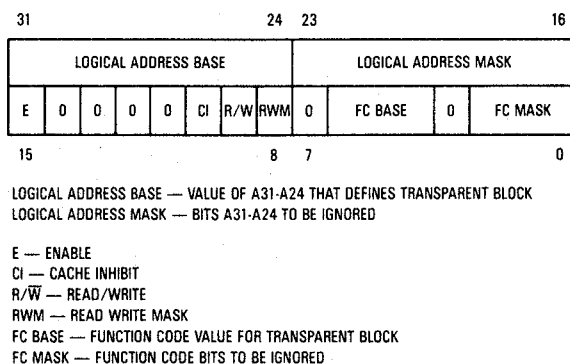
## 9

## Table Index (TIA, TIB, TIC, and TID)

These 4-bit fields specify the numbers of logical address bits used as the indexes for the four possible levels of the translation tables (not including the optional level indexed by the function codes). The index into the highest level table (following the function code, when used) is specified by TIA, and the lowest level, by TID. The fields contain integers, 0–15. When a zero value in a Tlx field is encountered during a table search operation, the search is over unless the indexed descriptor is a table (indirect) descriptor.

### 9.7.3 Transparent Translation Registers

The transparent translation registers (TT0 and TT1) are 32-bit registers that define blocks of logical address space that are transparently translated. Logical addresses in a transparently translated block are used as physical addresses, without modification and without protection checking. The minimum size block that can be defined by either TT<sub>x</sub> register is 16 Mbytes of logical address space. The two TT<sub>x</sub> registers can specify blocks that overlap. The TT<sub>x</sub> registers operate independently of the E bit in the TC register and the state of the MMUDIS signal. A transparent translation register is shown in Figure 9-37.



**Figure 9-37. Transparent Translation Register (TT0 and TT1) Format**

The fields of the transparent translation register are:

#### Enable (E)

This bit enables transparent translation of the block defined by this register:

0 — Transparent translation disabled

1 — Transparent translation enabled

A reset operation clears this bit.

### Cache Inhibit (CI)

This bit inhibits caching for the transparent block:

- 0 — Caching allowed
- 1 — Caching inhibited

When this bit is set, the contents of a matching address are not stored in the internal instruction or data cache. Additionally, the cache inhibit out signal ( $\overline{\text{CIOUT}}$ ) is asserted when this bit is set, and a matching address is accessed, signaling external caches to inhibit caching for those accesses.

### Read/Write (R/W)

This bit defines the type of access that is transparently translated (for a matching address):

- 0 — Write accesses transparent
- 1 — Read accesses transparent

### Read/Write Mask (RWM)

This bit masks the R/W field:

- 0 — R/W field used
- 1 — R/W field ignored

When RWM is set to one, both read and write accesses of a matching address are transparently translated. For transparent translation of read-modify-write cycles with matching addresses, RWM must be set to one. If the RWM bit equals zero, neither the read nor the write of any read-modify-write cycle is transparently translated with the TTx register.

9

### Function Code Base (FC BASE)

This 3-bit field defines the base function code for accesses to be transparently translated with this register. Addresses with function codes that match the FC BASE field (and are otherwise eligible) are transparently translated.

### Function Code Mask (FC MASK)

This 3-bit field contains a mask for the FC BASE field. Setting a bit in this field causes the corresponding bit of the FC BASE field to be ignored.

### LOGICAL ADDRESS BASE

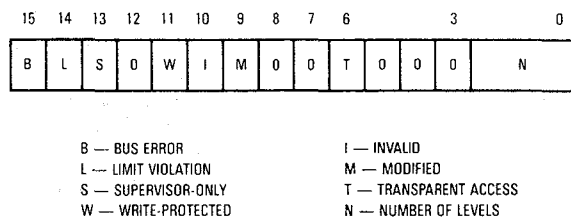
This 8-bit field is compared with address bits A31–A24. Addresses that match in this comparison (and are otherwise eligible) are transparently translated.

## LOGICAL ADDRESS MASK

This 8-bit field contains a mask for the LOGICAL ADDRESS BASE field. Setting a bit in this field causes the corresponding bit of the LOGICAL ADDRESS BASE field to be ignored. Blocks of memory larger than 16 Mbytes can be transparently translated by setting some of the logical address mask bits to ones. Normally, the low-order bits of this field are set to define contiguous blocks larger than 16 Mbytes, although this is not required.

### 9.7.4 MMU Status Register

The MMU status register (MMUSR) is a 16-bit register that contains the status information returned by execution of the PTEST instruction. The PTEST instruction searches either the ATC (PTEST with level 0) or the translation tables (PTEST with levels of 1–7) to determine status information about the translation of a specified logical address. The MMUSR is shown in Figure 9-38.



**Figure 9-38. MMU Status Register (MMUSR) Format**

The bits in the MMUSR have different meanings for the two kinds of PTEST instructions, as shown in Table 9-3.

**Table 9-3. MMUSR Bit Definitions**

| MMUSR Bit                | PTEST, Level 0                                                                                                                                                                                     | PTEST, Level 1–7                                                                                                                                                                                                                                                                 |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bus Error (B)            | This bit is set if the bus error bit is set in the ATC entry for the specified logical address.                                                                                                    | This bit is set if a bus error is encountered during the table search for the PTEST instruction.                                                                                                                                                                                 |
| Limit (L)                | This bit is cleared.                                                                                                                                                                               | This bit is set if an index exceeds a limit during the table search.                                                                                                                                                                                                             |
| Supervisor Violation (S) | This bit is cleared.                                                                                                                                                                               | This bit is set if the S bit of a long (S) format table descriptor or long format page descriptor encountered during the search is set, and the FC2 bit of the function code specified by the PTEST instruction is not equal to one. The S bit is undefined if the I bit is set. |
| Write Protected (W)      | This bit is set if the WP bit of the ATC entry is set. It is undefined if the I bit is set.                                                                                                        | This bit is set if a descriptor or page descriptor is encountered with the WP bit set during the table search. The W bit is undefined if the I bit is set.                                                                                                                       |
| Invalid (I)              | This bit indicates an invalid translation. The I bit is set if the translation for the specified logical address is not resident in the ATC or if the B bit of the corresponding ATC entry is set. | This bit indicates an invalid translation. The I bit is set if the DT field of a table or a page descriptor encountered during the search is set to invalid or if either the B or L bits of the MMUSR are set during the table search.                                           |
| Modified (M)             | This bit is set if the ATC entry corresponding to the specified address has the modified bit set. It is undefined if the I bit is set.                                                             | This bit is set if the page descriptor for the specified address has the modified bit set. It is undefined if I is set.                                                                                                                                                          |
| Transparent (T)          | This bit is set if a match occurred in either (or both) of the transparent translation registers (TT0 or TT1). If the T bit is set, all remaining MMUSR bits are undefined.                        | This bit is set to zero.                                                                                                                                                                                                                                                         |
| Number of Levels (N)     | This 3-bit field is cleared to zero.                                                                                                                                                               | This 3-bit field contains the actual number of tables accessed during the search.                                                                                                                                                                                                |

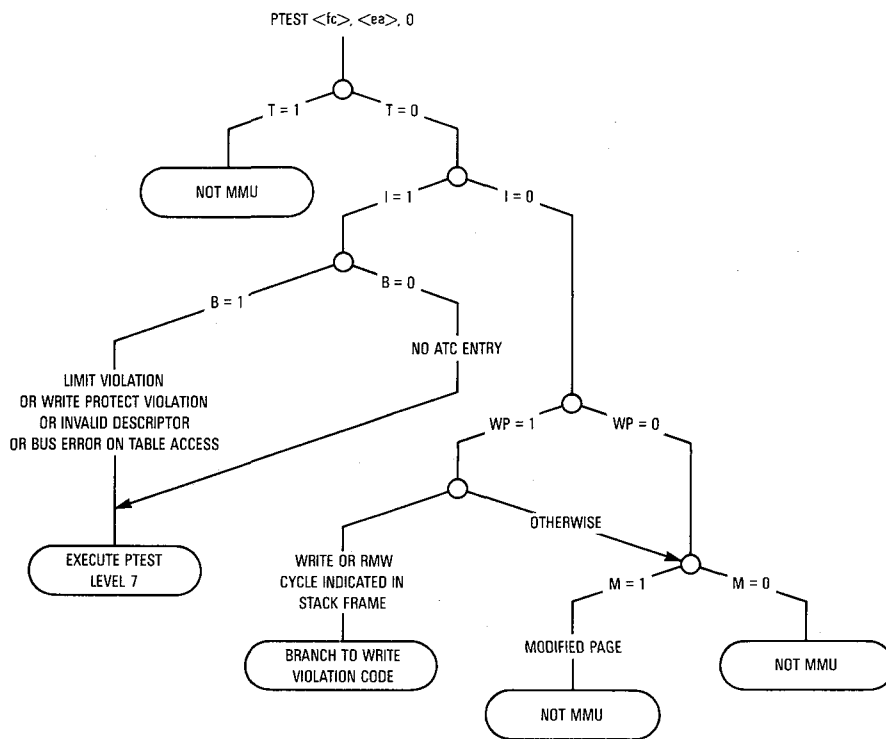
## 9.7.5 Register Programming Considerations

If the entries in the address translation cache (ATC) are no longer valid when a reset operation occurs, an explicit flush operation must be specified by the software. The assertion of  $\overline{\text{RESET}}$  disables translations by clearing the E bits of the TC and TT<sub>x</sub> registers, but it does not flush the ATC. Flushing of the ATC is optional under control of the FD bit of the PMOVE instruction that loads a new value into the SRP, CRP, TT0, TT1, or TC register.

The programmer of the MMU must be aware of effects resulting from loading certain registers. A subsequent section describes these effects. The MMUSR values lend themselves to the use of a case structure for branching to appropriate routines in a bus error handler. An example of a flowchart that implements this technique is shown in another section. A third section describes the conditions that result in MMU exceptions.

**9.7.5.1 REGISTER SIDE EFFECTS.** The PMOVE instruction is used to load or read any of the MMU registers (CRP, SRP, TC, MMUSR, TT0, and TT1). Since loading the root pointers, the translation control register, or the transparent translation registers with new values can cause some or all of the address translations to change, it may be desired to flush the ATC of its contents any time these registers are written. The opcodes of the PMOVE instructions that write to CRP, SRP, TC, TT0, and TT1 contain a flush disable (FD) bit that optionally flushes the ATC when these instructions are executed. If the FD bit equals one, the ATC is not flushed when the instruction is executed. If the FD bit equals zero, the ATC is flushed during the execution of the PMOVE instruction.

**9.7.5.2 MMU STATUS REGISTER DECODING.** The seven status bits in the MMU status register (MMUSR) indicate conditions to which the operating system should respond. In a typical bus error handler routine, the flows shown in Figures 9-39 and 9-40 can be used to determine the cause of an MMU fault. The PTEST instructions set the bits in the MMUSR appropriately, and the program can branch to the appropriate code segment for the condition. Figure 9-39 shows the flow for a PTEST instruction for the ATC (level 0), and Figure 9-40 shows the flow for a PTEST instruction that accesses an address translation tree (levels 1–7).

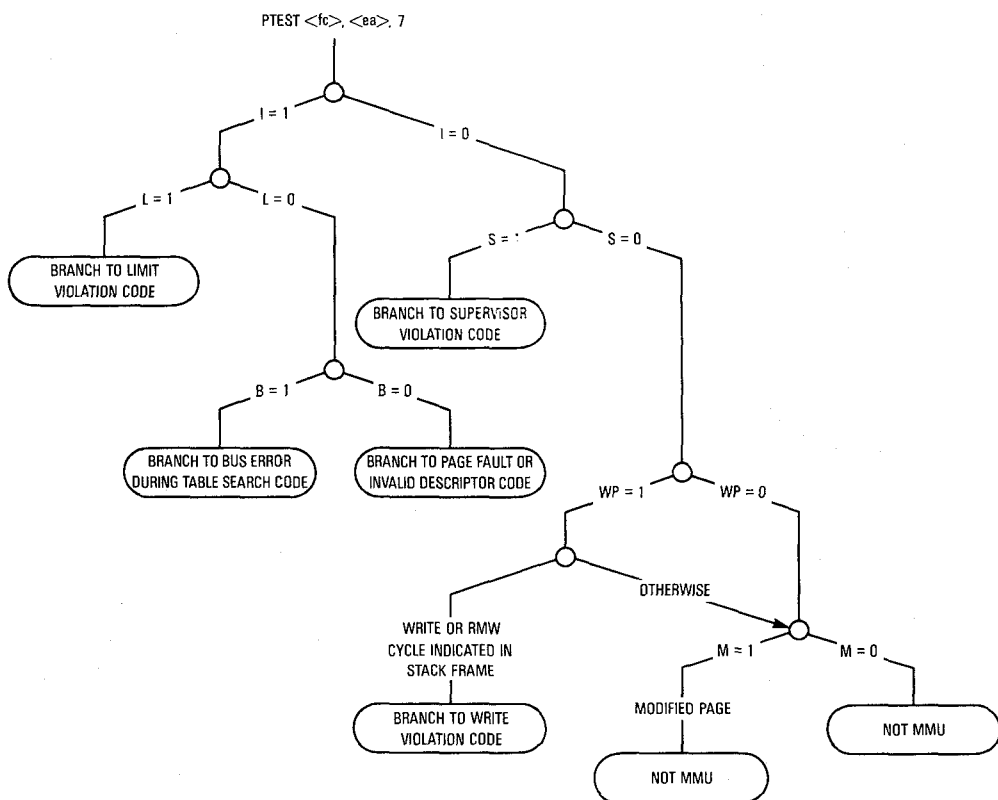


**Figure 9-39. MMU Status Interpretation PTEST Level 0**

**9.7.5.3 MMU CONFIGURATION EXCEPTION.** The exception vector table in the MC68030 assigns a vector for an MMU configuration error exception. The configuration exception occurs as the result of loading invalid data into the TC, SRP, or CRP register.

When the TC register is loaded with the E bit set, the MMU performs a consistency check of the values in all the four bit fields. The values in the Tlx fields are added until the first zero is encountered. The values in the PS and IS fields are added to the sum of the Tlx fields. If the sum is not equal to 32, the PMOVE instruction causes an MMU configuration exception. The instruction also causes a configuration exception when a reserved value (\$0–\$7) is placed in the PS field of the TC register.

A PMOVE instruction that loads either the CRP or the SRP causes an MMU configuration exception if the new value of the DT field is zero (invalid). In this case, the register is loaded with the new value before the exception is taken.



**Figure 9-40. MMU Status Interpretation PTEST Level 7**

## 9.8 MMU INSTRUCTIONS

The MC68030 instruction set includes four privileged instructions that perform MMU operations. A brief description of each of these instructions follows.

The PMOVE instruction transfers data between a CPU register or memory location and any one of the six MMU registers. The operating system uses the PMOVE instruction to control and monitor MMU operation by manipulating and reading these registers. Optionally, a PMOVE instruction flushes the ATC when it loads a value into the TC, SRP, CRP, TT0, or TT1 register.

The PFLUSH instruction flushes (invalidates) address translation descriptors in the ATC. PFLUSHA, a version of the PFLUSH instruction, flushes all entries. The PFLUSH instruction flushes all entries with a specified function code or the entry with a specified function code and logical address.

The PLOAD instruction performs a table search operation for a specified function code and logical address and then loads the translation for the address into the ATC. The operating system can use this instruction to initialize the ATC to minimize table searching during program execution. Any existing entry in the ATC that translates the specified address is flushed. The preload can be executed for either read or write attributes. If the write attribute is selected (PLOADW), the MC68030 performs the table search and updates all history information in the translation tables (used and modified bits) as if a write operation to that address had occurred. Similarly, if the read attribute is selected (PLOADR), the history information in the translation table (used bit) is updated as if a read operation had occurred. The PLOAD instruction does not alter the MMUSR.

The PTEST instruction either searches the ATC or performs a table search operation for a specified function code and logical address, and sets the appropriate bits in the MMUSR to indicate conditions encountered during the search. The physical address of the last descriptor fetched can be returned in an address register. The exception routines of the operating system can use this instruction to identify MMU faults. The PTEST instruction does not alter the ATC.

This instruction is primarily used in bus error handling routines. For example, if a bus error has occurred, the handler can execute an instruction such as:

```
PTESTW    #1,([A7, offset]),#7,A0
```

This instruction requests that the MC68030 search the translation tables for an address in user data space (#1) and examine protection information. This particular logical address is obtained from the exception stack frame ([A7, offset]). The MC68030 is instructed to search to the bottom of the table (#7 — there cannot be more than six levels) and return the physical address of the last table entry used in register A0. After executing this instruction, the handler can examine the MMUSR for the source of the fault and use A0 to access the last descriptor. Note that the PTESTR and PTESTW instructions have identical results except for PTEST0 when either TT<sub>x</sub> register matches the logical address and the R/W bit of that register is not masked.

The MMU instructions use the same opcodes and coprocessor identification (CpID) as the corresponding instructions of the MC68851. All F-line instructions with CpID=0 (including MC68851 instructions) that the MC68030 does



not support automatically cause F-line unimplemented instruction exceptions when their execution is attempted in the supervise mode. If execution of a unimplemented F-line instruction with CplD = 0 is attempted in the user mode, the MC68030 takes a privilege violation exception. F-line instructions with a CplD other than zero are executed as coprocessor instructions by the MC68030.

## 9.9 DEFINING AND USING PAGE TABLES IN AN OPERATING SYSTEM

Many factors must be considered when determining how to use the MMU in an operating system. The MC68030 provides the flexibility required to optimize an operating system for many system implementations. The example operating system described in the next section presents one approach to operating system design, with many of the tradeoffs discussed.

### 9.9.1 Root Pointer Registers

An operating system can use the CPU root pointer (CRP) register alone or both the CRP and the supervisor root pointer (SRP) registers to point to the top level address translation table(s). The choice depends on the complexity of the memory layout for the system. When only the CRP is used, it must point to a translation table that maps all supervisor and user references. However, the supervisor and user translation tables can be separate even when only the CRP register is used. When the index to the top level translation table is the function code value (FCL in TC register is set), supervisor and user tables are separate at all lower levels. With proper structuring of the address tables, both methods can provide the same functionality, but each has its advantages.

When the translation tables use the CRP and function code lookup, supervisor and user accesses are separate, and each task can have different supervisor and user mappings. Alternatively, the entries in the function code tables that correspond to the supervisor spaces for each task can all point to the same tables to provide a common mapping for all supervisor references.

When the mapping of the supervisor address space is identical for all tasks, the SRP can be used in conjunction with the CRP to provide a more simple and efficient way to define the mapping. This technique suppresses the use of the function code (unless the program and data spaces require distinct mappings) and separates supervisor and user accesses at the root pointer level of the translation tables. A single translation table maps all supervisor

accesses without maintaining a large number of supervisor pointers in the translation tables for each task, resulting in reduced bus activity for table searches.

## 9.9.2 Task Memory Map Definition

The MC68030 provides several different means by which the supervisor can access the user address spaces. The supervisor can access any user address, regardless of how the virtual space is partitioned, with the MOVES (move space) instruction. Some systems provide a complete 4-Gbyte virtual memory map for each task. Indeed, an operating system that runs other operating systems in a virtual machine environment must provide a complete map to accurately emulate the full addressing range for the subordinate operating system.

With the large address space of the MC68030, each individual user task or all user tasks can share the address space with the operating system. One method of performing this function is implemented in the example operating system in the next section. Sharing the address space provides direct access to user data items by the operating system. Another advantage of this mapping method is that tasks can easily share code. Common routines such as file I/O handlers and arithmetic conversion packages can be written re-entrantly and be restricted to read-only access from all user tasks in the system.

9

The simplest example of a shared virtual address space system is one in which each user and supervisor process is given a unique virtual address range within the single 4-Gbyte virtual address space. In other words, the system has only one linear virtual address space; all processes run somewhere in that space. Only one translation table tree is required for the entire system, but each task can have individual tables if desired. With the common tree approach, the operating system can access any item of any task without modifying the root pointer. Otherwise, only the currently active task is immediately accessible, which often is adequate. To switch tasks, the operating system only has to update the user program and user data pointers in the highest level translation table indexed by the function code. This gives each task access to its own data only. This scheme has the advantages of simple table management and easy sharing of common items by giving them the same virtual address for all tasks in the system. This scheme might be ideal for real-time systems that do not require more complexity in memory management facilities.

The next logical step toward increased operating system complexity, with shared user and supervisor virtual memory maps, is to keep the supervisor addresses separate but to give each user task its own use of the remainder of the virtual space. For example, each user task could have the virtual memory space from zero to 512 Mbytes; the operating system programs and data would occupy the remainder of the space, from 512 Mbytes up to 4 Gbytes. Each user task has its own set of translation tables. The supervisor root pointer may or may not be used, depending on whether the user tables also map the supervisor space. As in the preceding method, the user cannot access the operating system portion of the address space unless the operating system allows it or wishes to share common routines. The advantages of this scheme are that it provides a much larger virtual address space for each user task and it avoids virtual memory fragmentation problems. Disadvantages of this scheme include the requirement for slightly more complex table management and the restriction of operating system access to only the current user task.

There are few absolute rules in the use of the MC68030 MMU. In general, the statement regarding restricting operating system access to only one user task using the scheme described in the preceding paragraph holds true. However, by using the entire 4-Gbyte virtual address space and cross mapping the address space, the supervisor can access each user task space as a distinct portion of its own supervisor map. If each user task is limited to a 16-Mbyte virtual address space and the supervisor only requires a 16-Mbyte address space, 256 such address spaces can be mapped simultaneously. The supervisor translation tables can include each of these spaces, and the supervisor can access each task using indexed addressing with a register that contains the proper constant for a particular task. This constant provides a supervisor-to-user virtual address conversion. A systems programmer can implement some very sophisticated functions that exploit the flexibility of the MMU.

The most complex systems and those that implement virtual machine capability completely separate the virtual address spaces of the supervisor and all user tasks, or possibly even those of individual supervisor tasks. Each user or supervisor task has its own virtual memory space starting at zero and extending to 4 Gbytes. Using the function code, a 4-Gbyte address space for the program and another for its data can be provided for each task. Both the SRP and the CRP are probably used, since nothing is common among the various spaces. The operating system uses the MOVES instruction to interact with the user space. The advantages of this implementation are the maximum availability of the virtual space and a complete logical separation

or addresses. Virtual machine implementations require maximum availability of virtual space. The disadvantages are the more complex table management and the more restrictive accesses to other address spaces.

### 9.9.3 Impact of MMU Features on Table Definition

The features of the MMU that impact table definition are usually considered after deciding how to map memory for the tasks. For some systems, these features can affect the mapping decision and should be considered when making that decision.

**9.9.3.1 NUMBER OF TABLE LEVELS.** The MMU supports from zero to five levels (six levels with the use of indirection) in the address translation tables. The zero-level case is early termination at the root pointer. This provides a limit check on the range of physical addresses for the system. It is used primarily in systems that require the limit check on physical addresses.

Systems that support large page sizes or that require only limited amounts of virtual memory space can use single-level tables. A single-level translation tree with 32K-byte pages may be the best choice for systems that are primarily numerically intensive (i.e., the system is involved in arithmetic manipulations rather than data movement) where the overhead of virtual page faults and paging I/O must be minimized. This type of system can map a 16-Mbyte address space with only 2K bytes of page table space. With this much mapped address space, table search time becomes insignificant.

At another extreme is a single-user business system that only needs a 2-Mbyte virtual address space. A 512-byte page size might be best for this system, because the block size formats of many Winchester hard disk file systems is 512 bytes. A page table that completely maps the 2-Mbyte space requires only 16K bytes of memory, and the ATC entries directly map 11K bytes of virtual space at any one time. The page tables for this system and the one described in the preceding paragraph are small enough to be permanently allocated in the operating system data area. They incur virtually no management or swapping overhead.

A two-level address translation table provides a lower page level similar to the page tables in the two preceding paragraphs and additional direction at a higher level. For example, in a system using 32K-byte pages and 512-entry page tables, the upper level translation table contains 256 entries of short-format descriptors, requiring 1K bytes for the table. Each of the upper table

entries maps a 16-Mbyte region of the virtual address space. The primary advantage of a two-level table for large “number-crunching” system is the operating system designer’s ability to make a tradeoff between page size and table size. The system designer may choose a smaller page size to fit the block sizes on available I/O devices, yet keep the tables manageable. However, the designer must also consider the performance penalty associated with smaller page sizes. Systems with smaller page sizes have a higher frequency of page faults requiring more table search time and paging I/O. With the flexibility of the MC68030 MMU, the designer has enough choices to optimize table structure design and page size.

Three-level translation tables are useful when the operating system makes heavy use of shared memory spaces and/or shared page tables. Sophisticated systems often share translation tables or program and data areas defined at the page table level. When a table entry can point to a translation table also used by a different task, sharing memory areas becomes efficient. The direct access to user address space by the supervisor is an example of sharing memory.

Some artificial intelligence systems require very large virtual address spaces with only small fragments of memory allocated among these widely differing addresses. This fragmentation is due to the complex and recursive actions the system performs on lists of data. These actions require the system to constantly allocate and free sophisticated pointers and linked lists in the memory map. The fragmentation suggests a small page size to utilize memory most efficiently. However, small pages in a large virtual memory map require relatively large translation tables. For example, to map 4 Gbytes of virtual address space with 256-byte pages, the page tables alone require 64 Mbytes. With a three- or four-level table structure, the number of actual translation table entries can be drastically reduced. The designer can use invalid descriptors to represent blocks of unused addresses and the limit fields in valid descriptors to minimize the sizes of pointer and page tables. In addition, paging of the address tables themselves reduces memory requirements.

**9.9.3.2 INITIAL SHIFT COUNT.** The initial shift field (IS) of the translation control register (TC) can decrease the size of translation tables. When the required virtual address space can be addressed with fewer than 32 bits, the IS field reduces the size of the virtual address space by discarding the appropriate number of the most significant logical address bits. This technique inhibits

the system's ability to detect very large illegal (i.e., out-of-bounds) addresses. Using the full 32-bit address and reducing the table size with invalid descriptors and limited pointer and page table sizes prevents this problem.

**9.9.3.3 LIMIT FIELDS** Except for a table indexed by function code, every pointer and page table can have a defined limit on its size. Defining limits provides flexibility in the operating system and saves memory in the translation tables. The limit field of a table descriptor limits the size of the table to which it points. The limit can be either an upper or a lower limit, using either the lower or higher addresses within the range of the table. Since a task seldom requires the maximum number of possible virtual pages, this reduction in table size is practical.

For example, when an operating system uses 4K-byte pages and runs numerous small tasks that average 80K bytes each in size, each task requires a 20-entry page table. The system can limit the size of each table to 80 bytes, or 800 bytes for ten tasks. Without the limit, an operating system running ten of these tasks would require 40K bytes of space for the page tables alone (one table per page).

Memory savings required for translation tables is especially significant for artificial intelligence systems these systems tend to require very large memory maps. By using limit fields, each table is only as large as the number of active entries within it. This limit can change as the table grows. For higher level tables, each table only grows as the additional entries require. The use of three or four levels of tables facilitates the management of these tables.

**9.9.3.4 EARLY TERMINATION PAGE DESCRIPTORS.** A page descriptor residing in a pointer table is an early termination page descriptor mapping an entire block of pages. That is, it maps a contiguous range of virtual addresses to a contiguous range of physical addresses. For example, an operating system could reserve a 32K-byte area for special supervisor I/O peripheral devices. This area can be mapped with a single early termination descriptor to save translation table size and table search overhead. The descriptor can use the limit field to reduce the size of the contiguous block when the block size is smaller than the virtual address space that the particular descriptor represents. The MC68030 creates multiple ATC entries (one for each page) for the range of virtual addresses represented by the early termination descriptor as the pages are accessed.

An operating system can use an early termination page descriptor to map a contiguous block of memory for each task (both program and data). The tasks can be relocated by changing the physical address portion of the descriptor. This scheme is useful when the tasks in a system consist of one or a few sequential blocks of memory that can be swapped as a group. The operating system memory map can treat the entire address space within these blocks as a uniform virtual space available for all tasks. The system only requires one translation table; by the use of limit fields and early termination page descriptors, it maps complete segments of memory.

**9.9.3.5 INDIRECT DESCRIPTORS.** An indirect descriptor is a table descriptor residing in a page table. It points to another page descriptor in the translation tree. Using an indirect descriptor for a page makes the page common to several tasks. *History information for a common page is maintained in only one descriptor.* Access to the page sets the used (U) bit, and a write operation to the page sets the M (modified) bit for that page. When the operating system is searching for an available page, it simply checks the page table containing the descriptor for the common page to determine its status. With other methods of page sharing, the system would have to check page tables for all sharing tasks to determine the status of the common page.

**9.9.3.6 USING UNUSED DESCRIPTOR BITS.** In general, the bits in the unused fields of many types of descriptors are available to the operating system for its own purposes. The invalid descriptor, in particular, uses only two bits of the 32 (short) or 64 (long) bits available with that format. An operating system typically uses these fields for the software flags, indicating whether the virtual address space is allocated and whether an image resides on the paging device. Also, these fields often contain the physical address of the image.

The operating system often maintains information in an unused field about a page resident in memory. This information may be an aging counter or some other indication of the page's frequency of use. This information helps the operating system to identify the pages that are least likely to impact system performance if they are reallocated. The system should first use physical page frames that are not allocated to a virtual page. Next it should use pages with the longest time since the most recent access. Pages that do not have the M (modified) bit set should be taken first, since they do not need to be copied to the paging device (the existing image remains valid).

An aging counter can be set up in an unused field of a page descriptor. The system can periodically check the U (used) bit for the page and increment the count when the page has not been used since the previous check. The system can identify the least recently used page from the counts in the aging counter. When the counter for a page overflows, the system can list the page in a queue of least recently used pages from which it chooses the next page to be reallocated.

Many schemes afford the operating system designer a variety in selecting a page to be taken. One operating system scans page tables, starting at the lowest priority task, looking for aged pages to steal. Another system maintains a system-wide list of all page frames as they are used and scans the list, starting at the oldest, to find a page to steal. A sophisticated system keeps a working set model of active pages for each individual task. From this information, it can swap a complete block of pages in and out with a single I/O operation. The method chosen can have a dramatic impact on limiting page fault overhead in a heavily used system.

## 9.10 AN EXAMPLE OF PAGING IMPLEMENTATION IN AN OPERATING SYSTEM

This section describes an example operating system design that illustrates some of the MMU features. The description suggests alternatives to provide variations of the design. Memory management algorithms that can be implemented to derive the actual code are shown. A bus error handler routine is shown also. Implementing the algorithms develops the basic code for the memory management services of an operating system.

### 9.10.1 System Description

The example system has the ability to map a large virtual memory task space, which is required for execution of predominantly numerically intensive processing tasks. Most of these tasks do not need more than 16 Mbytes of memory, but the system can supply a larger virtual memory space (as large as 496 Mbytes) to the occasional task that requires more. The system uses the relatively large page size of 8K bytes to minimize thrashing and translation table searches. With a larger page size, fewer descriptors can map a large area of virtual memory. Also, in a given period of time, the MC68030 experiences fewer ATC misses and performs fewer table searches. The larger page size requires the paging I/O operations to transfer larger blocks of data, and sometimes only a small part of the page is actually used. However, preliminary software model simulations show that 8K-byte pages provide optimum performance for this type of processing.

The average task for this system is a compiler or text editor that requires only 192K bytes of memory, or 24 8K-byte pages. Using short page descriptors, the page table occupies 96 bytes.

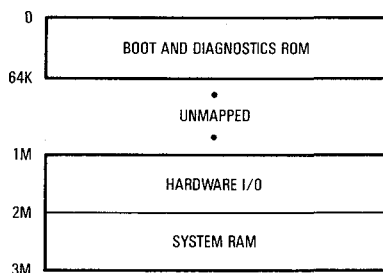
Page tables can reside at any 16-byte boundary; the limit fields of the MMU can provide the area needed without requiring excess space. This results in an address table area small enough to be completely resident in physical memory. The operating system does not need to page the table areas.

The paging hardware of many computer systems requires lower level tables to reside at page boundaries, effectively using one or more entire pages. This requires 80K bytes for the page tables for 10 tasks (10 tables, one 8K-byte page per table). Then, when the memory required for an upper level of tables is added, at a minimum of 8K bytes per task, the total comes to over 160K bytes. Table base addresses in the MC68030 are zero modulo 16 addresses. This results in a dramatic savings of memory for address table space; instead of using 80K bytes for the page tables for 10 tasks, (10 tables, one 8K-byte page per table), the MC68030 needs 960 bytes. Instead of 8K bytes per task for the upper level of tables, the tables require 2560 bytes in the MC68030. The fragmentation that may occur in allocating smaller tables could increase the memory requirement but would still remain less than 160K bytes.

The translation table tree for the example system consists of two levels. The upper level is a fixed table that contains 32 entries, each of which is a long-format table descriptor that points to a lower level page table. Each page table maps as many as 16 Mbytes of virtual address space. Since the upper level table is small (256 bytes), it can easily fit in the main control block of the task. When the system dispatches a new task, it loads a pointer to the upper level table for the task into the CRP register. Each lower level table consists of 0–2048 short-format page descriptors. The limit entry in the table descriptor for a page table determines the size of the table. For the average 192K-byte task, the upper level table usually has one valid entry, and this entry points to a lower level table with an average size of 96 bytes. A task that requires more than 16 Mbytes uses more than one valid entry in the higher level table.

In a typical computer system, with 64K bytes of boot and diagnostics ROM, a 64K I/O area, and 1 Mbyte of RAM, the physical mapping appears as follows:

LOW MEMORY



The operating system must control memory allocation for physical memory (page frames) to hold the pages of virtual memory. All available physical memory is divided into page frames, each of which can hold a page of virtual memory. A system with 4 Mbytes of actual memory is divided into 512 8K-byte frames that can theoretically hold 512 pages of active virtual memory at any one time. Usually, operating system components (exception handlers, the kernel, private memory pool) permanently reside in some of the memory. Only the remaining page frames are available for virtual memory pages.

The operating system maintains a linked list of all unallocated page frames. One simple way to do this is for each unallocated frame to contain a pointer to the next frame. The operating system takes the first page frame on the list when a frame is required. An operating system primitive called `GetFrame` performs this function and returns the physical address of an available frame. When all frames are allocated, `GetFrame` steals a frame from another task. `GetFrame` first looks for an unmodified frame to steal. An unmodified frame could be stolen without waiting for the page to be copied back to the external storage device that stores virtual page images. (This device is called the paging device or the backing store.) If no unmodified page frame is available, `GetFrame` must wait while the system copies a modified page to the paging device, then steals the page frame and returns to the caller with the physical address.

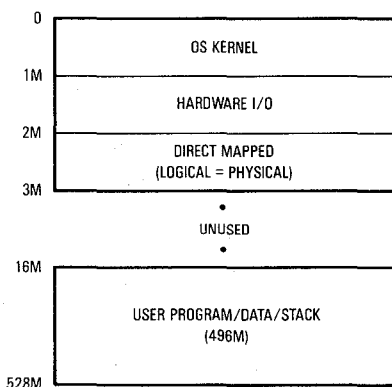
Next, the operating system needs physical memory management routines to allocate and free supervisor work memory. The routine must allocate pieces of memory on boundaries of at least modulo 16, the requirement for



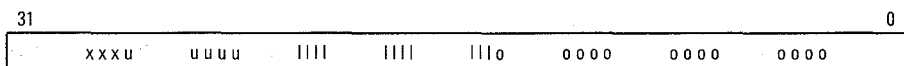
address translation tables. Typically, this type of routine allocates pieces of certain sizes. GetReal is the allocation routine; ReturnReal is the return routine. They use physical addresses.

With physical memory allocation provided for, the operating system must be able to manage virtual memory for all tasks. To do this, the system must be aware of the virtual memory map. It must know the total amount of virtual memory space, how much is allocated, and which areas are available to be assigned to tasks. The virtual memory map looks like this:

LOW MEMORY



Virtual addresses for this virtual memory are subdivided:



- x — Ignored (3 bits)
- u — Upper level index (5 bits), maps 32 long-table entries
- l — Lower level index (11 bits), maps 2048 short-page entries

The translation table structure consists of:

- CRP → upper level table in the task control block, which contains 32 long pointers:
- [0] → lower level table common to all tasks; maps all operating system areas (first 4 Mbytes of virtual space). This common table contains 512 short-page entries (2K bytes).
- [1] → lower level table for first 16 Mbytes of user program/data/stack area.
- .
- .
- .
- [31] → lower level table for last 16 Mbytes (of 496 total) of user program/data/stack area.

The user program can only access virtual addresses starting at 16 Mbytes and extending upward to the limit of 512 Mbytes. The code, the data, and the stacks for user tasks are allocated in this area of virtual memory. Supervisor programs can access the entire virtual map; they can access addresses that directly access the I/O ports as well as the entire physical memory at untranslated addresses. The address tables are set up so that virtual addresses are equal to the physical addresses for the supervisor between 1 and 3 Mbytes. Folding the physical address space into the virtual space greatly simplifies operations that use physical addresses. The folding does not necessarily mean that the virtual addresses are the same as the physical addresses. For example, the boot/diagnostic ROM at physical address zero could be assigned a virtual address of 3 Mbytes. However, any external bus masters or circuitry (such as breakpoint registers) resident on the physical side of the bus must have physical addresses. This requires the overhead of operating system code to perform address translation.

This virtual memory map provides supervisor addresses that are unique with respect to user addresses; all supervisor routines can directly access any user area without being restricted to certain instructions or addressing modes. The separate user and supervisor maps suggest that two root pointers should be used, one for the supervisor map and one for the user map. However, the supervisor must be able to access user translation tables for proper access to user data items. With separate root pointers, the supervisor table structure must be linked to that of the user. To do this requires an additional level of table lookup (function code level) for the supervisor address table.

This example uses a simpler scheme instead. Only the CPU root pointer is used, and, for each task, the first entry of the upper level table (for the supervisor portion, the first 16 Mbytes of virtual address space) points to the same lower level table. This common lower level table has supervisor pro-

tection and maps the entire virtual operating system, physical I/O, and physical memory areas. This scheme avoids the requirement for extra lookup levels or pointer manipulations during a task switch to furnish correct access across the user/supervisor boundary. All the operating system has to do when creating the address table for a new task is to set the first upper level table entry to point to the common page table of the supervisor.

To solve the problem of accounting for virtual memory areas assigned to a user task, the operating system uses the existing translation tables to identify these areas. When a valid descriptor points to a given virtual address page, this 8K-byte page of memory has been allocated. This scheme provides areas of memory that are multiples of the 8K-byte page size. Due to the 8K granularity, this scheme would be inadequate for tasks that continually request and return virtual memory space. As a result, some other technique would be used (perhaps auxiliary tables to show virtual space availability). The tasks in this system seldom request additional memory space; any request made is for a large area. This scheme suffices. The application programs and utilities that run in the UNIX (r) environment have similar requirements for memory.

The operating system primitive `GetVirtual` allocates virtual memory space for tasks. The input parameter is a block size, in bytes; `GetVirtual` returns the virtual address for the new block. `GetVirtual` first checks that the requested size is not too large. Then it scans the translation tables looking for an unallocated virtual memory area large enough to hold the requested block. If it does not find enough space, `GetVirtual` attempts to increase the page table size to its maximum. If this does not provide the space, `GetVirtual` returns an error indication. When the routine finds enough virtual space for the block, it sets the page descriptors for the block to virgin status (invalid, but allocated). When these pages are first used, a page fault is generated. The operating system allocates a page frame for the page and replaces the descriptor with a valid page descriptor. The status (indicated by a software flag in the invalid descriptor) tells the operating system that the paging device does not have a page image for this page; no read operation from the paging device is required.

When the status of an invalid descriptor indicates that a page image must be read in, primitive `SwapInPage`, reads in the image. The input parameter for this routine is the invalid descriptor, which contains the disk address of the page image. Before returning, `SwapInPage` replaces the invalid descriptor with a valid page descriptor that contains the page address. The page is now ready for use.

These routines provide many of the functions required for the memory management services of an operating system, but a complete memory management system requires a complementary function for each routine. The complementary function usually performs the same steps in the reverse order. The complement of `GetVirtual` could be `ReturnVirtual`; for `SwapInPage`, the complement might be `SwapOutPage`. These counterparts can be derived to perform similar steps in the reverse order.

## 9.10.2 Allocation Routines

This section describes the central routine `Vallocate`, which user programs call to obtain memory. In this section (and the next), a loose high-level language syntax is used for the code. The code takes many liberties to enhance readability. For example, the code assigns descriptive strings for return status values. These strings typically represent binary values. Also, the code uses empty brackets to represent obvious subscripts in loops that scan tables. In such a loop, the subscript on the second line is obvious:

```
for Upper_Table_Index = 1 to 31 do
    if Upper_Table [Upper_Table_Index].Status = invalid then ...
```

In the code shown here, the second line is:

```
    if Upper_Table [].Status = invalid then ...
```

The code uses flag operations that are assumed to be defined elsewhere in the system. They may imply more complex operations than bit manipulations. For example, page table status of invalid virgin can be implemented with an invalid descriptor instead of the page descriptor, and a software flag bit in the descriptor that indicates the page is allocated but has never been used (the paging device has no page image).

`Vallocate` has a single input parameter, the required memory size in bytes. It returns status information and the virtual address of the start of the area (if the memory is allocated). To simplify the routine, it always returns a multiple of the system page size and never allocates a block that crosses a 16-Mbyte boundary. It could allocate a portion of a page by implementing a control structure to subdivide a page, but, if the control structure were within the allocated page, the user could corrupt it. The block could cross a 16-Mbyte boundary if the routine included code to keep track of consecutive free blocks when scanning the lower level tables, each of which represents 16 Mbytes of address space. Once the total area is located, `Vallocate` allocates the consecutive blocks and returns the address of the lowest block.

The 32 upper level table entries are long pointer types; each represents 16 Mbytes of virtual address space. Each entry is either invalid (has no lower page tables) or allocated (has lower page tables and a limit field that defines

the table size). By convention, the first entry maps the supervisor address space and has supervisor protection. The routine never modifies this first entry. The 31 entries after the first are available to be allocated as user address space.

A routine similar to this that linearly extends (grows) a previously allocated memory block could be written. A stack is a good example. The operating system can allocate the top of the memory (the thirty-second upper level table entry) as a stack that grows downward from the highest address. If a task needs several large stacks, a 16-Mbyte block can be used for each stack, with a software flag set to indicate growth in a downward direction.

The logic of Vallocate is:

1. Validate the request and calculate number of pages required.
2. Scan each upper table entry's lower page tables (where they exist) looking for an adequate group of unallocated pages.
3. If no space is found, see if the lower table is less than its maximum size and if the block can be allocated by expanding it at the end.
4. If still no space is found, use the next free upper table entry and initialize its new lower level page table to allocate the block here.
5. Set allocated page entries to indicate virgin status (allocated, invalid, and not swapped out).
6. Return status. If status is OK, also return virtual address.

The code for Vallocate is:

**Vallocate (SizeInBytes, VirtualAddressReturned, Status);**

```

/* The following are global to all routines */
/* Symbolically define the upper level pointer table */

Declare Upper_Table[32] Record of
    Status=(unallocated, allocated), /* lower table here or not */
    Limit_Field=(0 to 4k),           /* limit for lower page table */
    Pointer;                         /* address of lower page table if allocated */

/* Symbolically define the lower level page table */

Declare Lower_Table[0 to Limit_Field] Based Record of
    Status=(invalid_unallocated, /*not allocated to User */
            invalid_paged_out,   /*allocated but paged out */
            invalid_virgin,      /*allocated but not yet used */
            valid_in_memory),    /*allocated and in memory */
    Pointer;                     /*physical address or disk address of page */

```

```

Declare Upper_Table_Index, Lower_Level_Index;      /*table indexes          */

Declare NumPages;                                  /* number of pages required to hold request */

Status = "Out of virtual Memory";                  /* default result status to this error      */

if SizeInBytes > 16 megabytes then exit Vallocate;

NumPages = (SizeInBytes+PageSize-1)/PageSize;      /* Pages needed          */

/* Scan User eligible page tables                  */

for Upper_Table_Index = 1 to 31 do
    If Upper_Table[Index].Status = allocated then call SearchPageTable;
    If Status = "OK" then Exit Vallocate;
end;

/* Block not found so find upper level entry unallocated and call SearchPageTable that will 'expand' */
/* the null table to hold the block.                */

for Upper_Table_Index = 1 to 31
    If Upper_Table[Index].Status = unallocated then call SearchPageTable;

/* No more virtual space, exit leaving Status = "out of virtual memory" */

exit Vallocate;

Procedure SearchPageTable;

/* Scan table pointed to by upper level index to see if it can hold the block. If not, see if it can be */
/* be expanded. If successful then set flags in the page entries, set status to "OK" and User's */
/* virtual address */

    Declare Maxfound;                                /* Count of consecutive free blocks found */

    Maxfound = 0;
    For Lower_Level_Index = 0 to Upper_Table[Index].Limit_Field

        /* count consecutive free pages until Maxfound met or not */
        If Lower_Table[Index].Status = invalid_unallocated then do
            Maxfound = Maxfound+1;
            if Maxfound >= NumPages then do

```

```

/* Found! Now flag the page entries, update the MC68030 and */
/* return the User's virtual address */
while (Maxfound > 0) do
    Lower_Table[].Status = invalid_virgin;
    Lower_Level_Index = Lower_Level_Index-1;
end;

Status = "OK";
VirtualAddressReturned =
    Upper_Level_Index*16Meg +
    Lower_Level_Index*8k;
PLOAD (VirtualAddressReturned);
exit SearchPageTables;
end;

end;

/* allocated page hit so start counting from zero again */
else Maxfound = 0;

/* If we get here there was not room. See if we can expand the page table to hold the new block */
/* If so grow it and set the new page entries as virgin */

If Upper_Table[].Limit + NumPages < 4k then do
    NewLimit = Upper_Table[].Limit + NumPages;

    /* We can grow the page table! First get area for new table */
    Call GetReal(4*NewLimit, NewPageTable);

    /* Now copy the first part of the old table into the new */
    for Lower_Table_Index = 0 to Upper_Table[].Limit
        NewPageTable->Lower_Table[] = Lower_Table[]

    /* Return the old table and install the new table pointer */
    Call ReturnReal(4*Upper_Table[].Limit, Upper_Table[].Pointer);

Upper_Table[].Pointer = NewPageTable;

    /* Set returned virtual address and load it replacing the old */
    VirtualAddressReturned = Upper_Level_Index*16Meg + Lower_Level_Index*8k;
    PLOAD (VirtualAddressReturned) /* refresh MC68030 */

    /* Set all the new entries at the end to virgin status */
    While (Lower_Table_Index < NewLimit) do
        Lower_Table_Index = Lower_Table_Index + 1;
        Lower_Table[].Status = invalid_virgin;
    end;

    /* Set OK status and return with it */
    Status = "OK";
    exit SearchPageTables;
end;

/* cannot expand the table. return with status unchanged (failed) */
end SearchPageTables;

```

### 9.10.3 Bus Error Handler Routine

The routine that processes bus error exceptions is the most critical part of the memory management services provided by the example operating system. This routine must determine the validity of page faults and perform the necessary processing. It must identify the conditions that aborted the executing task. The PTEST instruction can investigate the cause of a bus error by performing a table search using the address and type of access that produced the error, accumulating status information during the search.

When the PTEST instruction does not find any error, the bus error was most likely a malfunction (for example, a transient memory failure). The operating system must respond appropriately.

The table search performed by the PTEST instruction may end in a bus error termination. Either the address translation tables are not correctly built or main memory has failed (either a transient or permanent failure).

A supervisor protection violation or a write protection violation usually indicates that the task generating the exception attempted to access an area of the virtual address space that is not part of the task's address space. The operating system usually recovers from such an error by terminating (aborting) the task.

9

When the PTEST instruction returns the invalid status, the bus error is a page fault, and the operating system must identify the specific type of page fault. When the limit violation bit returned by the PTEST instruction is set, the task that took the exception was trying to access a page that has not been allocated. The example system aborts the task in this case. In other systems, this is an implicit request for more virtual memory, particularly if the reference is in a stack area.

When no limit violation occurred, a descriptor is invalid. Typically, the descriptor contains software flags that provide relevant information. The example operating system checks to see if the invalid descriptor is in an upper level or a lower level table. When the descriptor is in the upper level table, the task was attempting to access unallocated virtual memory, and the system aborts the task. When the descriptor is in a lower level table, the system checks software flags to identify the invalid descriptor.

When the software flags indicate that the descriptor corresponds to an unallocated page, the system aborts the task. When the descriptor refers to a virgin page (allocated, but not yet accessed) and the request for the page

was a read request, the page is actually invalid because the read operation reads unknown data. However, the example operating system does not consider the type of request, but assigns a physical page frame to the page and writes the page descriptor to the page table. Some systems clear virgin pages to zero.

When the software flags indicate that the page is allocated and the image has been copied to the paging device, the operating system assigns a page frame, reads the page image into the frame, and writes the page descriptor to the page table. Another possible type of invalid descriptor is one that requires special processing, such as one that refers to a virtual I/O device area in a virtual machine.

Obtaining a page frame for a virtual page may be an obvious operation. However, when no idle page frame is available, the system must steal one. If the page in the stolen frame has been modified in memory, the system must save the page image on the paging device. The system must alter the translation table of the task that loses the frame to show that the page is allocated and swapped out. Typically, the translation table entry shows the address of the page image on the paging device.

The method a system uses to select a page frame to steal varies a great deal from system to system. A simple system may just steal a page from the lowest priority task. More advanced systems select the page frame that has not been accessed for the longest time. This process, called aging, is done in several ways. One method uses bits of the page descriptor as an aging counter. Periodically, the operating system examines the U (used) bits and increments the count for pages that have not been used. The system maintains a list of pages with aging counters that have overflowed. The pages on this list are available for stealing.

Some systems keep a separate list of pages that have not been modified since the page image was read from memory. The page frames that contain these pages can be stolen without swapping out because the existing page image on the paging device remains valid.

Page stealing software can involve many dynamics of the system. It can consider task priority, I/O activity, working-set determinations, the number of executing tasks, a thrashing level, and other factors.

The example bus error exception routine is called `BusErrorHandler`. It is more general than `Vallocate` because it relies on several operating-system-dependent items. The variable pointer `VictimTask` is assumed to point to a table from a task that is losing a page frame. This assumption is necessary because control block layout and the method of searching for and finding other tasks in the example operating system are not defined. The code is further simplified by omitting the function code value and the read/write status, which do not affect the basic logic of the program.

```

/* Paging Bus Error Handler for example O.S. */

Procedure BusErrorHandler (BusErrAddress);

/* Global Variables to all code */

Declare TableEntry; /*Pointer returned by PTEST instruction
                    /* pointing to the lowest level entry in the
                    /* translation tables.

/* Use MC68030 PTEST instruction to get fault status and table entry
case PTEST (BusErrAddress,TableEntry) of

    /* Bus Error - translation table is invalid or memory hardware problems. Terminate the task.
    B: AbortTask("Invalid table or memory hardware error");

    /* Supervisor Violation - task tried accessing restricted memory
    S: AbortTask("Attempted access of Supervisor-only memory");

    /* Write Protected - tried writing into read-only memory
    W: AbortTask("Attempted write into read-only memory");

    /* Limit Violation - tried accessing unmapped virtual space. This happens in our example
    /* O.S. when accessing within a 16 megabyte segment in User memory past what is
    /* currently allocated for the lower page table as determined by the upper level limit field.
    L: AbortTask("Invalid address");

    /* Invalid - pointer indicates invalid. Must determine status.
    I: begin

        /* If upper level entry then that 16 Meg chunk of the virtual space is unallocated
        /* and has no page tables.
        If TableEntry is upper level then AbortTask("Invalid address");

        /* We are at a page table entry. Look at software flags.

        /* If this page unallocated to the User then abort task
        If EntryStatus=invalid_unallocated then
            AbortTask("Invalid Address");

        /* If this page is virgin then assign to it a physical frame
        if EntryStatus = invalid_virgin then do
            GetFrame(TableEntry); /* address returned in entry
            PLOAD (BusErrAddress); /* update MC68030 entry
            exit BusErrorHandler; /* done so continue task
            end do;

```

```

/* If this page is swapped out then read it back in */
if EntryStatus = invalid_swapped_out then do
    /* first get a frame to hold the new page */
    DiskAddress = TableEntry.Pointer;          /* disk location */

    GetFrame(TableEntry);                      /* address returned in entry */

    /* Now read in the virtual page image */
    call SwapPageIn(TableEntry,DiskAddress);
    PLOAD (BusErrAddress);                    /* update MC68851 entry */
    exit BusErrorHandler;                      /* done so continue task */
end do;

end begin;

/* No MC68030 status bits on. Must be memory malfunction or RMW cycle with no
/* ATC entry
Otherwise:    If Stack_Frame shows RMW instruction (SSW) then
               /* ATC did not have descriptor loaded and MC68030 cannot
               /* search tables to load it. Explicitly load it and allow the task to
               /* continue normally

               Begin
                   PLOAD (BusErrAddress); /* update ATC
                   exit BueErrorHandler; /* done so re-execute instruction
               end Begin

               Else: AbortTask("Memory Malfunction");

end case;

```

#### Procedure GetFrame(FrameTableEntry);

```

/* This module returns the address of a physical frame in the passed table entry. It obtains one
/* from the free frame list. If none there it scans a queue pointing to pages that have been
/* recorded as having aged by not being accessed frequently. It first tries to find a read-only
/* page in the queue but if none it returns the first (oldest) entry after swapping the page out
/* to disk and altering the translation tables of the owning task. If nothing in the queue it waits
/* for some other task to free a frame by terminating or deallocating memory

Restart:
    if Free_Frame_Queue NOT null then
        Dequeue first entry and return its value.

    if Aged_Frame_Queue NOT null then begin

        /* First try to find a read-only page
        If scanning finds read-only page then use and dequeue it
        else dequeue the first entry (which is the oldest);

        Find owning task and the frames current page entry;

        /* Invalidate owning task's page
        PFLUSH (User_Space,VictimTask.VirtualAddress);

```

```

/* If modified page swap it out. SwapPageOut either gives control to other tasks */
/* during the I/O or copies the page returning immediately. */
If modified then call SwapPageOut(VictimTask.TableEntry);
/* Disk address now in Victim's page entry */

```

```

/* Now set the old task's page status and return the frame */
VictimTask.TableEntry.Status = invalid_swapped_out;
return physical frame value;
end do;

```

```

/* At this point we can use some other stealing method but we just wait until another task frees */
/* a frame by terminating or freeing memory. */
call wait (Free_Frame);
go to Restart;

```

```

end GetFrame;

```

```

Procedure SwapPageIn (SwapInTableEntry,DiskAddress);

```

```

/* This procedure takes the disk address and reads the page from the paging external media */
/* into the physical address residing in the table entry pointer. */
end SwapPageIn;

```

```

Procedure SwapPageOut(SwapoutTableEntry);

```

```

/* This procedure performs output on the external paging device and then replaces the */
/* physical page frame address in the page entry pointer field with the disk address of the */
/* block holding the image of the page. */
end SwapPageOut;

```

```

Procedure AbortTask(TerminationMsg);

```

```

/* This procedure terminates the current task and issues a diagnostic message. */
end AbortTask;

```

```

end BusErrorHandler;

```